



Data-driven surrogate modeling framework for real-time prediction of airfoil lookup tables: application to proprotor optimization

Yu-Eop Kang^{1,2} · Dawoon Lee² · Yoonpyo Hong² · Sunwoong Yang^{3,4} · Kwanjung Yee⁵ 

Received: 23 June 2025 / Revised: 26 November 2025 / Accepted: 30 November 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

Optimization of blade shapes plays a key role in enhancing the aerodynamic efficiency of rotorcraft. However, blade-shape optimization remains challenging due to the strong spanwise variation in local flow conditions experienced by each blade section during rotation. Designers typically employ low-fidelity aerodynamic solvers in rotor blade design due to their computational efficiency. However, these solvers rely on pre-computed airfoil performance lookup tables, making it difficult to treat airfoil shapes as design variables during optimization, thus limiting design flexibility. While surrogate modeling offers a promising solution for real-time generation of airfoil lookup tables, several challenges remain due to the need to account for diverse flow conditions across a broad airfoil design space. To address these challenges, this study proposes an efficient deep-neural-network framework, named *Airfoil Brain*, which integrates data-driven airfoil parameterization, scalable and reliable surrogate modeling, and an adaptive sampling strategy. Consequently, this framework allows simultaneous consideration of both airfoil and planform parameters in rotor blade-shape optimization. The effectiveness of the proposed framework is demonstrated through a multi-objective aerodynamic optimization of a proprotor.

Keywords Airfoil parameterization · Surrogate modeling · Adaptive sampling · Blade optimization

1 Introduction

During the operation of rotor blades and propellers, each blade section encounters various flow conditions along its radial location due to differences in rotating speed, chord

length distribution and twist angle distribution. Accordingly, it is essential to optimize sectional airfoil geometries, along with the chord and twist distributions, in order to adapt to these local flow conditions and achieve high efficiency. To this end, it is crucial to develop an efficient strategy capable of exploring a wide airfoil design space. Leishman and Rosen (2011); Hong et al. (2024).

In the conceptual design stage, designers typically rely on low-fidelity analyses such as blade element momentum theory (BEMT) (Goldstein 1929) and lifting-line-theory-based analyses (Johnson 1998; Hodges et al. 2007; Saberi et al. 2004; Bauchau et al. 2001). To reduce computational complexity, these methods assume two-dimensional flow and utilize pre-computed aerodynamic performance lookup tables, commonly known as C81 tables (Jain et al. 2016). They offer a balanced trade-off between computational efficiency and accuracy, allowing rapid design optimization during the initial design phase.

However, a major bottleneck of these low-fidelity methods is the time-consuming process of preparing C81 tables, which must encompass a wide range of Mach number (Ma), angle of attack (α) and Reynolds number (Re) combinations. This inherent complexity makes it infeasible to consider airfoil

Responsible Editor: Boris Krämer

Work conducted while the author was affiliated with the Department of Aerospace Engineering, Seoul National University.

✉ Kwanjung Yee
kjyee@snu.ac.kr

¹ Manufacturing AI Research Center, Korea Institute of Industrial Technology, Incheon 21999, Republic of Korea

² Department of Aerospace Engineering, Seoul National University, Seoul 08826, Republic of Korea

³ Machine Technology Research Institute, Korea Advanced Institute of Science and Technology, Daejeon 34051, Republic of Korea

⁴ Department of Mechanical Engineering, Hanyang University, Ansan 15588, Republic of Korea

⁵ Institute of Advanced Aerospace Technology, Seoul National University, Seoul 08826, Republic of Korea

shapes as design variables during the optimization process. Consequently, engineers typically employ a single airfoil shape or a limited set of predetermined airfoil shapes across the radial span of the blades, and then optimize only plan-form parameters such as twist and chord distributions. The design constraints substantially restrict the design flexibility, potentially resulting in sub-optimal designs.

To address this limitation, a novel approach to rapidly evaluate C81 tables of diverse airfoil geometries is essential. One potential solution is the use of surrogate modeling, which is a data-driven approach that creates a “cheap-to-evaluate” function to approximate the high-fidelity function. With such models, the aerodynamic performance of unseen airfoils can be efficiently predicted, thereby enabling direct incorporation of airfoil geometries as design variables in the rotor blade optimization process. Nevertheless, since C81 tables must cover a wide range of airfoil geometries and flow conditions, two key challenges remain: (1) the compact representation of diverse airfoil geometries and (2) the computational complexity of constructing large training datasets that span a wide range of flow regimes.

The first challenge arises from the fact that the airfoil design space must be broad enough to encompass optimal geometries for various flow regimes potentially encountered by rotor blades. However, traditional parameterization methods, including PARSEC (Sobieczky 1999), class/shape-function transformation (CST) (Kulfan 2008), and B-splines/Bézier curve representation (Piegl and Tiller 1997; Derksen and Rogalsky 2010), require a larger number of design variables to accurately represent the geometry (flexibility) (Masters et al. 2017).

One state-of-the-art solutions is the data-driven parameterization method, such as singular value decomposition (SVD) (Toal et al. 2010; Li et al. 2019), generative adversarial networks (GANs) (Goodfellow et al. 2020; Chen et al. 2020; Du et al. 2020) and Variational Autoencoders (VAEs) (Kingma and Welling 2013; Xie et al. 2024; Kang et al. 2023; Yang et al. 2023). These methods consider the parameterization task as the feature extraction problem, where latent features are learned from large-scale airfoil datasets such as the UIUC airfoil database (Selig 1996). By extracting compact and meaningful low-dimensional features from high-dimensional airfoil coordinates, they represent an extensive airfoil design space efficiently while preserving geometric flexibility (Masters et al. 2017).

Despite these advancements, data-driven approaches often lack feasibility, as the generated samples tend to have noisy and self-intersecting curves (Achour et al. 2020; Yilmaz and German 2020). This results in a process requiring manual intervention within the entire optimization loop, which prevents a fully automated optimization framework. In addition, the feature extraction processes are usually abstract and difficult to interpret, making the learned design variables

less intuitive and unexplainable. This lack of interpretability makes it difficult to intuitively define design space boundaries, resulting in an overly large design space that degrades the optimization efficiency. Hence, it is crucial to improve the feasibility and intuitiveness of data-driven parameterization methods, while achieving a balanced trade-off between the flexibility and compactness of design variables.

The second challenge arises because C81 tables must cover a wide range of flow regimes. This diversity leads to response surfaces that exhibit highly localized and distinct behaviors: for instance, transonic regimes exhibit strong nonlinearities within a narrow range of angles of attack; subsonic regimes display relatively linear behavior over a wider range; whereas flow regimes dominated by unsteady effects — such as those involving large-scale flow separation — yield noisy response surfaces due to the inherently chaotic nature of the unsteady flow and the amplification of numerical noise (Gilkeson et al. 2014; Han et al. 2012).

The diversity of flow regimes in the C81 tables implies that training samples must be strategically distributed for efficient surrogate modeling. However, conventional sampling techniques, such as Latin hypercube sampling (LHS), distribute samples uniformly across the entire domain, making them ineffective for capturing localized nonlinearities. To overcome this limitation, adaptive sampling techniques can be employed, which iteratively add new samples based on the knowledge acquired from earlier iterations (Fuhg et al. 2021). Recent adaptive sampling frameworks are predominantly based on Gaussian Process regression (GPR) or Kriging, where the predictive uncertainty can be analytically derived and directly used as the acquisition criterion (Kyprioti et al. 2020; Liu et al. 2016).

While GPR-based adaptive sampling has shown great success for small- to medium-scale problems, its cubic computational complexity (Rasmussen 2003) makes it infeasible for large-scale tasks such as C81 table prediction, which typically involve more than 10^6 training samples (Li et al. 2019; Li and Zhang 2021; Healy et al. 2024).

Neural networks (NNs) offer a practical alternative owing to their ability to efficiently handle large datasets (Bouhlel et al. 2020; Du et al. 2021; Li et al. 2019; Li and Zhang 2021; Shalu et al. 2023; Safdar et al. 2024; Anand et al. 2024). However, standard NNs do not inherently provide predictive uncertainty, making direct extensions to adaptive sampling difficult. As a result, existing NN-based aerodynamic surrogate models remain limited to relatively narrow angle-of-attack ranges (Li et al. 2019; Du et al. 2021) or subsonic conditions (Shalu et al. 2023; Safdar et al. 2024; Anand et al. 2024). In the broader machine learning community, various techniques have been developed to quantify predictive uncertainty in NNs — such as Monte Carlo dropout (Gal and Ghahramani 2016), deep ensembles (Lakshminarayanan et al. 2017), and Bayesian NNs (Blundell et al. 2015). How-

ever, their application to adaptive sampling for regression tasks remains limited, and to the best of our knowledge, NN-based adaptive sampling has not yet been explored for the C81 table prediction problem.

In this study, we propose a comprehensive deep-neural-network-based framework, named *Airfoil Brain*, to address the aforementioned challenges in predicting C81 tables. The framework integrates data-driven parameterization, scalable and reliable NN-based surrogate modeling, and an adaptive sampling method to efficiently predict C81 tables across a wide range of airfoil geometries. The key contributions of this study can be summarized as follows:

1. A large design space of airfoil geometries is efficiently parameterized using a data-driven method. Unlike existing GAN- or SVD-based methods that often generate self-intersecting curves, the proposed method ensures geometric feasibility while providing compact and physically interpretable design variables.
2. An uncertainty-aware adaptive sampling framework is developed by integrating a scalable deep ensemble surrogate model. In contrast to conventional uniform sampling or GPR-based adaptive methods limited by scalability, the proposed framework efficiently scales to the large dataset, while capturing localized nonlinearities across diverse flow regimes.
3. The proposed framework, *Airfoil Brain*, enables real-time prediction of C81 tables and allows the concurrent optimization of both airfoil and planform parameters, thereby enhancing rotor blade aerodynamic performance.

The remainder of this paper is organized as follows: Section 2 overviews the proposed framework, followed by a detailed description of its main methodology. Section 3 presents the training and validation of the framework using high-fidelity CFD simulation. Finally, Section 4 demonstrates the framework's applicability for aerodynamic shape optimization of a propeller.

2 Methodology

2.1 Framework overview

This study proposes the *Airfoil Brain* framework to efficiently predict the C81 tables for a wide range of airfoil geometries. The key challenges in building this framework are to address (1) how to represent various airfoil geometries with compact design variables and (2) how to train the surrogate model with minimal computational cost. To this end, two core neural networks are developed: the Airfoil Generation Network (AGN), which provides a compact and intuitive representation of airfoil geometries through a data-driven

parameterization, and the Performance Prediction Network (PPN), which serves as a surrogate model that enables rapid aerodynamic predictions along with predictive uncertainties. An adaptive sampling method is also integrated to iteratively train the PPN, thereby enabling it to better capture the characteristics of the response surface and improve computational efficiency during data preparation.

Figure 1 presents a schematic overview of how each component is combined to iteratively train the model. First, airfoil geometries are sampled for data preparation. This is efficiently achieved by sampling through compact and intuitive latent variables provided by the AGN. Second, high-fidelity CFD simulations are carried out to evaluate the aerodynamic performance of the airfoil geometries under various flow conditions, thereby constructing the training database. This yields a training dataset, where the inputs are the latent variables and flow conditions (Ma , Re , and α), and the outputs are the aerodynamic coefficients (C_l , C_d , C_m). Next, the surrogate model (PPN) is trained on this dataset. DE is employed because it is scalable to large datasets and capable of providing uncertainty estimates. These estimates are then exploited through adaptive sampling, which selects additional samples in regions of high uncertainty to further improve the model's accuracy. This process is iteratively repeated until the model reaches the desired accuracy or the computational budget is exhausted.

After training, the aerodynamic performance of arbitrary airfoil geometries can be rapidly evaluated without additional expensive CFD computations. This significantly accelerates the airfoil lookup table generation process required for rotor blade analysis, enabling airfoil shapes to be considered as design variables. Consequently, the framework supports the concurrent optimization of airfoil and planform parameters in propeller design, as demonstrated in Section 4. Details of the AGN, PPN, and adaptive sampling strategy are presented in the following subsections.

2.2 Airfoil generation network (AGN)

To explore a broad range of the airfoil design space efficiently, it is essential to represent high-dimensional airfoil geometries using compact design variables. Data-driven parameterization methods offer a promising solution by extracting underlying geometric features from large airfoil databases and utilizing them as design variables. Several approaches — such as SVD (Toal et al. 2010; Li et al. 2019), GAN (Chen et al. 2020; Du et al. 2020), and VAE (Xie et al. 2024; Kang et al. 2023; Yang et al. 2023) — have been proposed for this purpose. However, SVD, being a linear decomposition method, is inherently limited in capturing nonlinear geometric variations, while GAN-based models often suffer from unstable training and mode collapse (Goodfellow et al. 2020; Arjovsky et al. 2017). In contrast, VAE

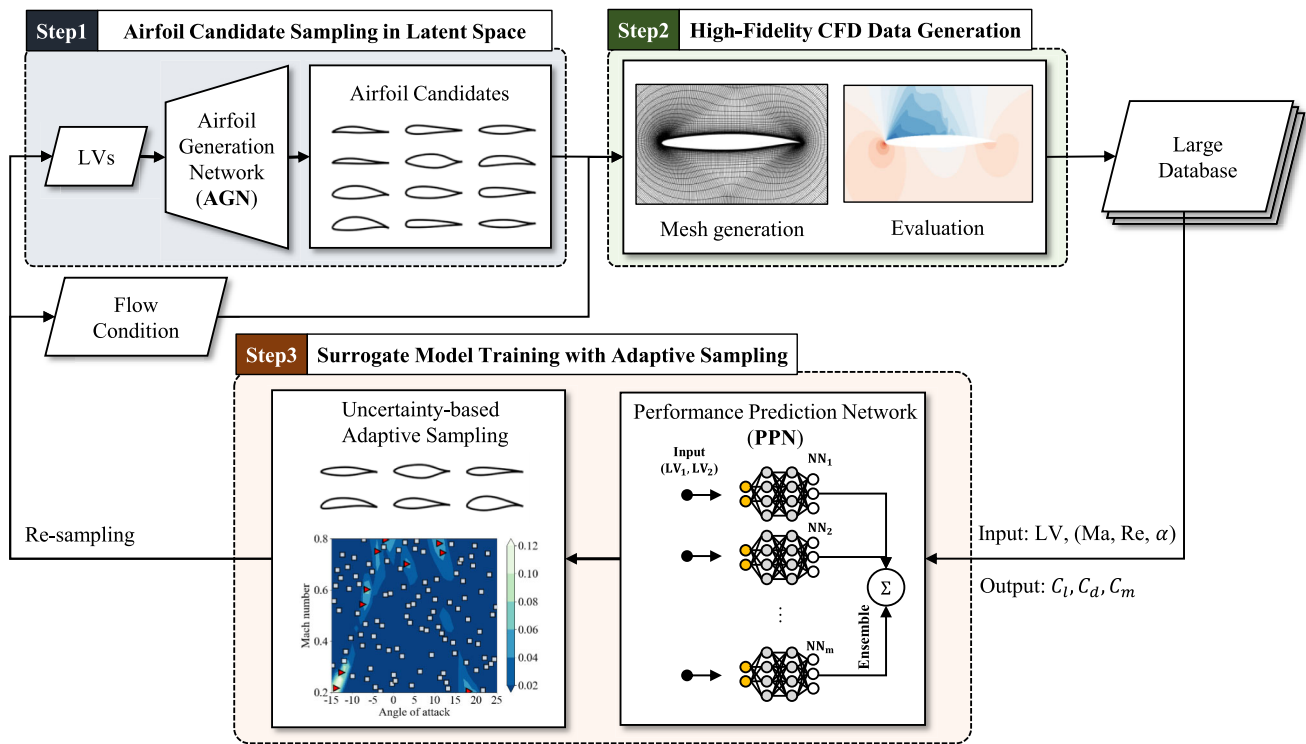


Fig. 1 Schematic overview of the training process of *Airfoil Brain*. The process consists of three main stages: (1) latent space sampling to generate airfoil candidates, (2) high-fidelity CFD simulations to evaluate aerodynamic performance under various flow conditions, and (3)

surrogate model training with uncertainty-guided adaptive sampling. Once trained, *Airfoil Brain* enables rapid prediction of aerodynamic coefficients for arbitrary airfoil geometries

enables stable convergence owing to its likelihood-based loss function with explicit regularization of the latent distribution, ensuring a smooth and continuous latent space (Kingma and Welling 2013; Higgins et al. 2016). In this study, a VAE-based airfoil parameterization method (Kang et al. 2023, 2024) is adopted as baseline architecture and further customized to enhance its intuitiveness and geometric feasibility.

In general, the loss function of vanilla VAE is defined as follows:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\frac{1}{N_s} \|\mathbf{x} - \mathbf{x}'\|_2^2}_{\text{Reconstruction loss}} + \underbrace{\beta \cdot D_{\text{KL}}(q(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{\text{Regularization loss}} \quad (1)$$

where $\mathbf{x}$ and $\mathbf{x}'$ are the original and reconstructed samples, $\mathbf{z}$ is the latent variables, N_s is the number of samples, and $D_{\text{KL}}(\cdot)$ is the Kullback-Leibler divergence. The loss function consists of two terms: reconstruction and regularization losses. The reconstruction loss encourages the decoder to accurately reconstruct the original input from latent variables, while the regularization loss forces the encoder's output distribution to align with a predefined prior, typically a Gaussian distribution. The balance between the two loss terms is controlled by the hyperparameter β . Since the latent space is regularized to follow the Gaussian prior, new sam-

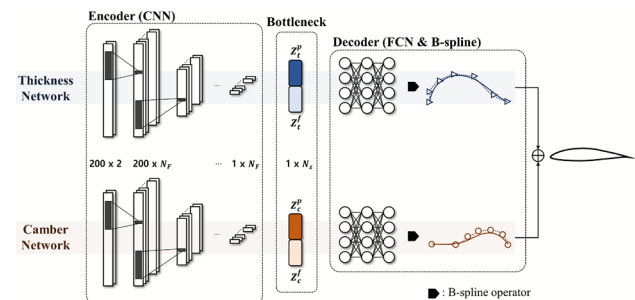


Fig. 2 Model architecture of Airfoil Generation Network (Kang et al. 2023)

ples can be efficiently generated by sampling from the prior distribution. This characteristic facilitates the efficient generation of realistic samples for downstream applications such as design optimization.

The AGN is also based on the VAE architecture, but it is adapted for airfoil parameterization to improve feasibility and interpretability (Kang et al. 2023), where feasibility means generating geometrically valid airfoils without self-intersection, and interpretability means representing latent variables that are well aligned with the physical features of the airfoil. As shown in Fig. 2, the AGN consists of two encoders: one for the compression of thickness distribution and another for the compression of camber distribution.

This “two-way” structure, which separates the thickness and camber generations, is adopted for two main reasons. First, because the latent spaces for the thickness and camber distributions are separate, users can control both distributions independently, which is an intuitive approach aerodynamically. Second, by ensuring that the thickness distribution remains positive, the generation of self-intersecting airfoils can be completely avoided — a property that is not guaranteed when the airfoil x – y coordinates are directly generated.

The decoding process is also adapted to improve feasibility: the model output is constrained by B-spline layers. Instead of directly generating the thickness and camber distributions, it first generates the control points of the B-splines, which are then used to reconstruct the thickness and camber distributions. This structure ensures the generation of smooth and continuous curves, which are desirable in aerodynamic design.

Another key feature of the AGN is that the latent variables are directly aligned with physical features of an airfoil to improve intuitiveness. To achieve this, as illustrated in Fig. 2, the latent variables are first divided into physical latent variables z^p and free latent variables z^f . The physical latent variables are trained to be aligned with user-defined physical features, while the free latent variables learn auxiliary features that are not captured by the physical latent variables. This is achieved by introducing a physics loss term, defined as follows:

$$\mathcal{L}_{\text{AGN}} = \mathcal{L}_{\text{VAE}} + \lambda \cdot (\|z^p - z_0^p\|_2^2) \quad (2)$$

where z^p denotes the physical latent variables that are aligned with the physical features of the airfoil z_0^p , and λ is the weight parameter that balances the loss terms. λ is set to 10^{-5} , ensuring that the physical loss term remains sufficiently small and does not dominate the overall training objective (typically contributing less than 5% to the total loss). By minimizing the difference between the physical features and the latent variables, the latent variables are directly aligned with the physical features. This alignment enhances the interpretability of the airfoil parameterization.

The AGN uses the UIUC airfoil database (Selig 1996) as training samples. For physical features, maximum thickness (t_{max}), maximum camber (m_{max}), leading edge radius ($r_{\text{L.E.}}$), and trailing edge direction ($\gamma_{\text{T.E.}}$) were selected as physical features, and each was aligned to its corresponding physics latent variables of AGN: $z_{t_{\text{max}}}$, $z_{m_{\text{max}}}$, $z_{r_{\text{L.E.}}}$, and $z_{\gamma_{\text{T.E.}}}$. The remaining free latent variables are trained to represent auxiliary features not captured by the physical latent variables.

For the detailed model architecture of the AGN, a one-dimensional convolutional NN (CNN) is used for the encoder and fully connected NN (FCN) for the decoder. To further improve the performance of the encoder, several key hyperparameters were carefully chosen: the size of the fil-

ter (N_{filter}) and kernel (N_{kernel}), the dimension of the latent space for thickness and camber (N_{latent}), the activation functions, and β . The optimal hyperparameter settings were chosen using a standard grid search procedure (Bishop and Nasrabadi 2006), resulting in the following configuration: $N_{\text{kernel}} = 7$, $N_{\text{filter}} = 32$, $N_{\text{latent}} = 12$, GELU activation, and $\beta = 2.5 \times 10^{-8}$ (see Appendix B.1).

For the training settings, the Adam algorithm was selected as the optimizer, which is used in deep learning due to its stable and efficient convergence characteristics (Kingma 2014). The initial learning rate was set to 10^{-2} and decayed by a factor of 0.7 every 2,500 epochs, for a total of 25,000 training epochs. While the latent space was set to 24 dimensions, the model naturally suppressed many of them during training, resulting in an effective latent dimension of 10. This indicates that the model automatically identified and ignored redundant dimensions, resulting in a compact and efficient latent representation. For more details, please refer to the Appendix B.1.

The airfoil design space trained by AGN is visualized in Fig. 3. The left sub-figure represents the $z_{m_{\text{max}}} - z_{t_{\text{max}}}$ plane, and right sub-figures represent the $z_{\gamma_{\text{T.E.}}} - z_{r_{\text{L.E.}}}$ plane conditioned by $z_{m_{\text{max}}}$, $z_{t_{\text{max}}}$. The physical latent variables are shown to independently control their corresponding airfoil features, enabling a clear and intuitive definition of the airfoil design space boundaries. The quantitative analysis of the representation capability of AGN is presented in Appendix C.1.

2.3 Performance prediction network (PPN)

To efficiently construct the surrogate model for predicting C81 tables of unseen airfoil geometries, the PPN must meet the two key requirements: (1) the scalability to handle a large database and (2) the ability to quantify predictive uncertainty to guide the adaptive sampling process.

In the context of uncertainty quantification, two sources of uncertainty are considered: data uncertainty (aleatoric) and model uncertainty (epistemic). Data uncertainty arises from numerical variability in the CFD data, such as numerical noise or non-converged RANS solutions, and represents an irreducible component of uncertainty. Model uncertainty, on the other hand, originates from the surrogate model itself due to limited or uneven data coverage, and can be reduced through additional sampling or model refinement.

Given the complexity of various flow regimes, which lead to noisy and highly nonlinear response surfaces, it is desirable to consider both data and model uncertainties. Hence, this study employs DE, an ensemble-based neural network framework capable of estimating both types of uncertainty (Lakshminarayanan et al. 2017).

DE consists of multiple probabilistic NNs, each of which represents a conditional probability distribution $p(y|\mathbf{x})$, and combines these distributions to produce the final prediction.

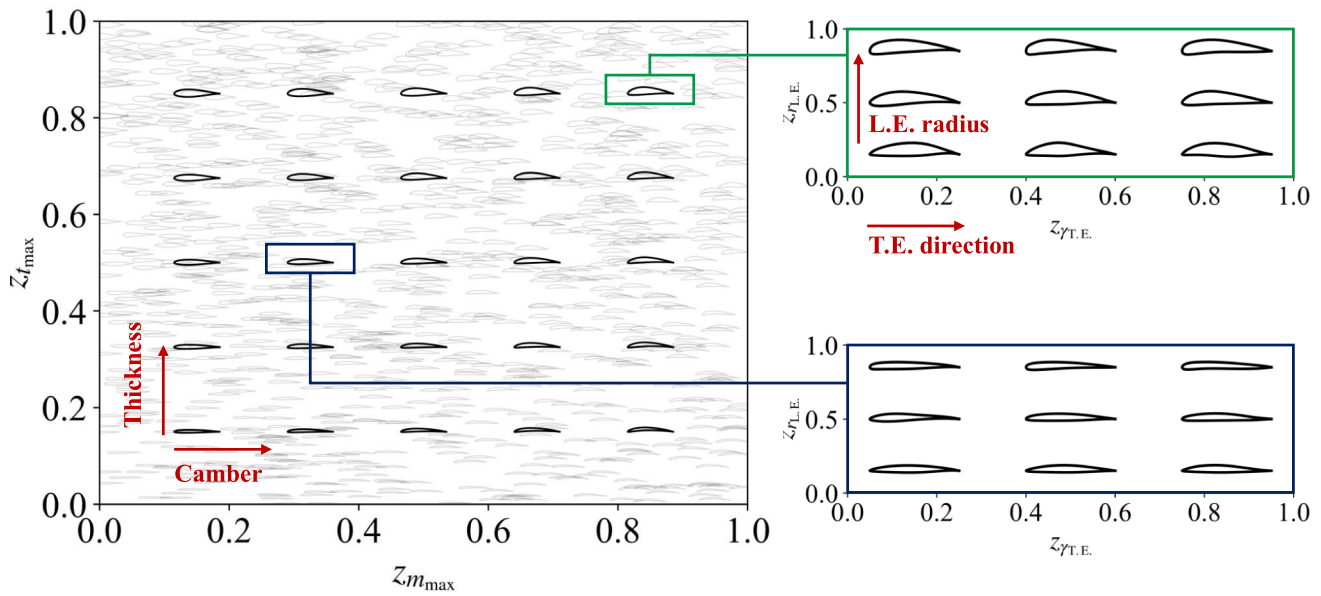


Fig. 3 Visualization of the latent space defined by the airfoil generation network. The left figure shows airfoil geometries projected onto the plane spanned by maximum thickness ($z_{t,max}$) and maximum camber ($z_{m,max}$).

The blue and green boxes indicate the subspaces where leading edge radius ($z_{r_{L,E}}$) and trailing edge angle ($z_{\gamma_{T,E}}$) are varied, respectively, while maximum thickness and camber remain constant

Figure 4 illustrates the difference between a vanilla NN and a probabilistic NN. Unlike a vanilla NN that deterministically predicts the output quantity $\mathbf{y}$, a probabilistic NN treats each observation as a realization of an underlying probability distribution and learns to model this distribution. In general, the output is assumed to follow a Gaussian distribution parameterized by a mean $\mu(\mathbf{x})$ and the variance $\sigma^2(\mathbf{x})$, such that $y \sim \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x}))$. By learning the entire distribution rather than a single point value, probabilistic NN can quantify data uncertainty.

This difference in output formulation also leads to a different training objective. While the vanilla NNs are often trained by minimizing the mean-squared-error (MSE) of the training samples, probabilistic NNs are trained by minimizing the negative-log-likelihood (NLL) (Lakshminarayanan et al. 2017). The MSE and the NLL for Gaussian distribution is defined in Eq. 3 and 4, respectively.

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N_s} \sum_{i=1}^{N_s} (y_i - \hat{y}(\mathbf{x}_i))^2 \quad (3)$$

$$\mathcal{L}_{\text{NLL}} = \frac{1}{N_s} \sum_{i=1}^{N_s} \left[\frac{\log(\sigma^2(\mathbf{x}_i))}{2} + \frac{(y_i - \mu(\mathbf{x}_i))^2}{2\sigma^2(\mathbf{x}_i)} + \frac{\log(2\pi)}{2} \right] \quad (4)$$

where N_s denotes the number of training samples, $\mathbf{x}_i$ and y_i represent the input and target output of the i -th sample, respectively, $\hat{y}(\mathbf{x}_i)$ is the deterministic prediction of the

vanilla NN, $\mu(\mathbf{x}_i)$ and $\sigma^2(\mathbf{x}_i)$ denote the predicted mean and variance of the probabilistic NN, respectively.

The intuitive interpretation of the NLL can be found in Guo et al. (2017); Kendall and Gal (2017): if the MSE term $(y_i - \mu(\mathbf{x}_i))^2$ is high at $\mathbf{x}_i$, then the probabilistic NN is trained to increase $\sigma^2(\mathbf{x}_i)$ to reduce the term $\frac{(y_i - \mu(\mathbf{x}_i))^2}{2\sigma^2(\mathbf{x}_i)}$. Conversely, if the MSE is low at $\mathbf{x}_i$, the model is trained to decrease $\sigma^2(\mathbf{x}_i)$. This means that the $\sigma^2(\mathbf{x}_i)$ term automatically distinguishes whether the region has high or low MSE. In fact, the loss function for the probabilistic NN in Eq. 4 can be reduced to the loss function for a vanilla NN in Eq. 3, by assuming a constant σ . This implies that a probabilistic NN is a generalization of a vanilla NN that can consider non-homogeneous predictive variance across the input parameter domain.

While a probabilistic NN can effectively capture data uncertainty, reflecting the inherent noise in the data, it cannot represent model uncertainty, which refers to the functional variability arising from limited training data or randomness in the training process. To account for this source of uncertainty, a DE trains multiple probabilistic NNs under slightly different conditions (e.g., random initialization or data sampling), thereby exploring diverse functional representations of the same problem. By aggregating these independently trained models, the DE captures both data and model uncertainty. As shown in Fig. 5, a total of M probabilistic NNs are trained independently, each producing a Gaussian distribution. The final predictive distribution is then expressed as a mixture of Gaussian distributions, as defined by the following equations.

Fig. 4 Comparison of neural network architectures: **a** vanilla neural network, **b** probabilistic neural network

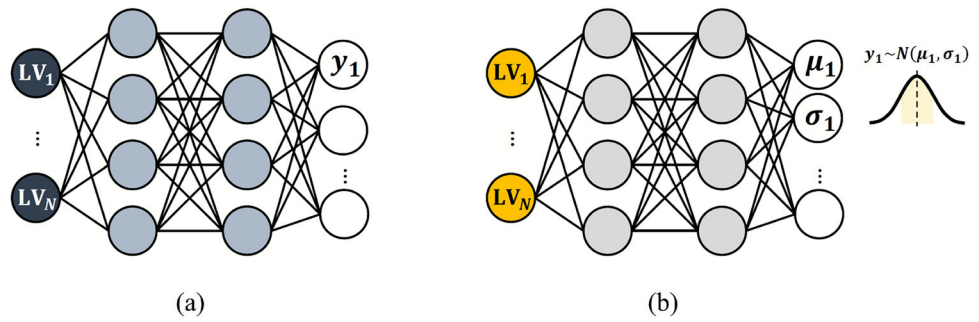
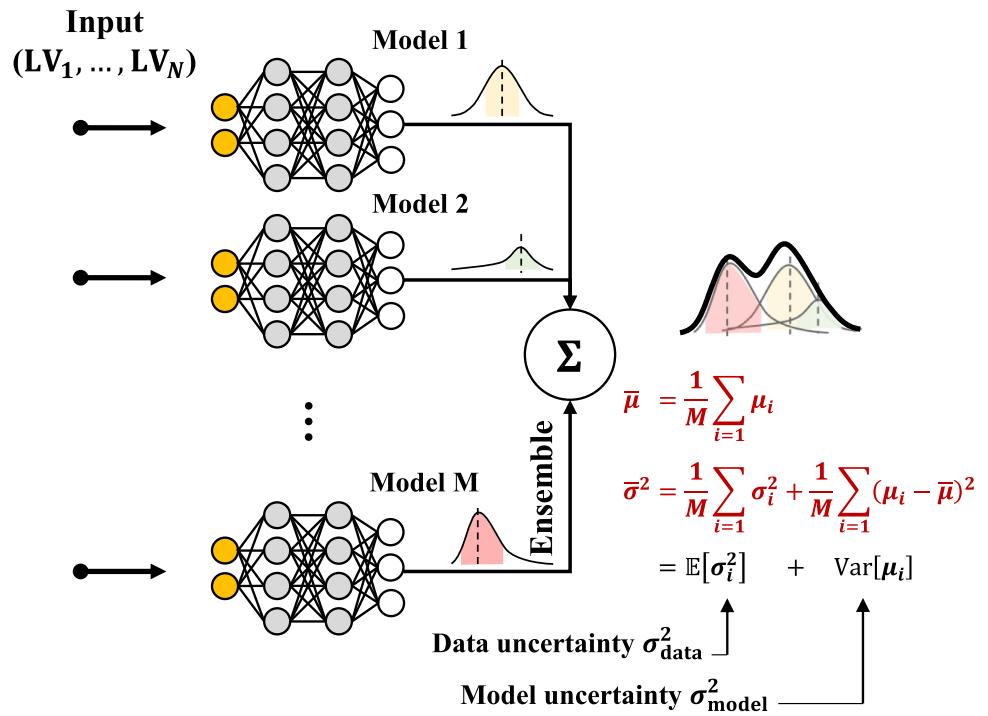


Fig. 5 The architecture of Deep ensemble. Multiple probabilistic neural networks are independently trained to model the conditional Gaussian distribution of the output $p(y|\mathbf{x})$. Deep ensemble then aggregates these individual distributions into a Gaussian mixture to estimate the final predictive mean $\bar{\mu}$ and uncertainty $\bar{\sigma}^2$. The total predictive uncertainty is decomposed into data (aleatoric) uncertainty and model (epistemic) uncertainty



$$\bar{\mu} = \frac{1}{M} \sum_{i=1}^M \mu_i \quad (5)$$

$$\begin{aligned} \bar{\sigma}^2 &= \frac{1}{M} \sum_{i=1}^M \sigma_i^2 + \frac{1}{M} \sum_{i=1}^M (\mu_i - \bar{\mu})^2 \\ &= \underbrace{\mathbb{E}[\sigma_i^2]}_{\text{Data uncertainty}} + \underbrace{\text{Var}[\mu_i]}_{\text{Model uncertainty}} \end{aligned} \quad (6)$$

where M denotes the number of probabilistic NN in the ensemble, μ_i and σ_i^2 represent the predicted mean and variance from the i -th probabilistic NN, respectively, $\bar{\mu}$ is the ensemble-averaged mean computed as the arithmetic mean of μ_i , and $\bar{\sigma}^2$ denotes the total predictive variance that can be decomposed into the data uncertainty $\mathbb{E}[\sigma_i^2]$ and the model uncertainty $\text{Var}[\mu_i]$.

Throughout this process, DE can quantify both data and model uncertainties. A noteworthy point in the training of DE is that multiple models should be diverse to accurately

quantify model variability. Conventionally, techniques such as bagging and bootstrapping are used to promote diversity among models during training. However, in NNs, it has been shown that the random initialization of NN weights is sufficient to introduce diversity, even when trained on identical datasets (Lakshminarayanan et al. 2017). Moreover, the number of probabilistic NN is also an important factor affecting predictive uncertainty. Following the recommendation of Lakshminarayanan et al. (2017), we set $M = 5$ in this study. However, this configuration does not necessarily ensure optimal uncertainty quantification across different problems. Therefore, we further calibrated the uncertainty using a standard deviation (STD) calibration method proposed by Yang and Yee (2024).

2.4 Uncertainty-guided adaptive sampling

Since the C81 tables cover a broad range of flow regimes, their response surfaces exhibit varying degrees of nonlin-

earity — nearly linear behavior in attached-flow regimes and highly nonlinear behavior near unsteady or transonic regions. Under these conditions, conventional sampling methods such as LHS become inefficient, as they distribute samples uniformly without accounting for localized nonlinearities. Therefore, adaptive sampling becomes essential, as it progressively refines the model by allocating more samples to regions of highly nonlinear behavior.

To this end, it is necessary to define an appropriate score function. Predictive uncertainty is a widely used criterion, as the regions exhibiting higher uncertainty generally have larger prediction errors, — a relationship further supported by the cross-validation results presented in Section 3.2.3. Furthermore, DE enables the decomposition of predictive uncertainty into data and model uncertainties, as described in Section 2.3. To fully exploit this aspect, we propose the following novel score function $\mathcal{S}(\mathbf{x})$:

$$\mathcal{S}(\mathbf{x}) = \max(\sigma_{\text{model}}(\mathbf{x}) - \alpha_w \sigma_{\text{data}}(\mathbf{x}), 0) \quad (7)$$

where $\sigma_{\text{model}}(\mathbf{x})$ and $\sigma_{\text{data}}(\mathbf{x})$ denote the standard deviations corresponding to the model uncertainty and the data uncertainty, respectively, and α_w is a weighting coefficient controlling the relative contribution of the data uncertainty to the score function.

The design of this score function is based on the principle that high model uncertainty indicates high model error and is therefore more likely to benefit from additional samples. In contrast, high data uncertainty reflects inherent noise in the data itself, where prediction errors are irreducible and additional sampling provides little benefit. Therefore, from the perspective of flow regimes, it is desirable to add new samples in the transonic and steady regimes, where nonlinearity is significant. Conversely, allocating samples to highly unsteady regimes, even in the presence of large errors, is less effective due to dominant data uncertainty. In summary, the score function allocates more samples to regions where the model has not yet sufficiently learned the underlying response (high

model uncertainty), while avoiding regions where inherent data noise limits the potential benefit of additional samples (high data uncertainty). The weight parameter α_w is introduced to balance these two criteria. In this study, this value was set to 0.5 through simple numerical experiments presented in Section 3.1.

During the iterations of adaptive sampling, samples with a high score function are selected and evaluated as new training samples. To fully exploit parallel computing resources, batch sampling, which selects multiple samples simultaneously at each iteration, is strongly recommended. However, naively selecting the top-ranked samples based solely on the score function tends to concentrate new samples within a narrow region (as shown by the red triangle markers in Fig. 6a), thereby reducing the effectiveness of batch sampling.

Algorithm 1 Batch-sampling strategy using k-means clustering

Input: Training dataset D , deep ensemble model DE, number of new samples N_{add}

Output: New samples for next iteration x_{new}

1. Select random samples of $\mathbf{x}_{\text{rand}}$ from the input space.
 2. Calculate the score function $\mathcal{S}(\mathbf{x}_{\text{rand}})$.
 3. Sort the scores $\mathcal{S}(\mathbf{x}_{\text{rand}})$ in descending order and extract the top 10% of the samples $\hat{\mathbf{x}}$.
 4. Perform K-means clustering on the extracted samples, with $k = N_{\text{add}}$.
 - minimize $\sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - c_j\|^2$
 - where c_j is the centroid of the j -th cluster C_j
 5. Obtain x_{new} from the k centroids of clusters c_j .
-

To address this issue, this study proposes a k-means clustering-based batch-sampling approach. The detailed procedure is presented in Fig. 6b and Algorithm 1. First, random samples are generated from the input space, and the score function is evaluated. The high-score candidates, belonging to the top 10% of the score function, are selected, as indi-

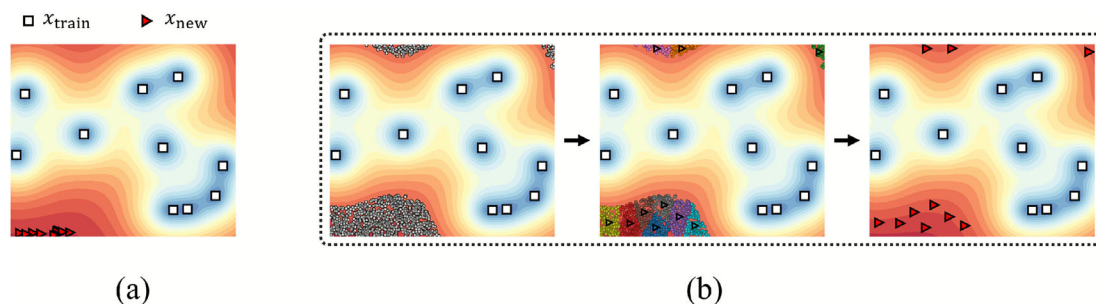


Fig. 6 Schematic illustrations of batch-sampling strategies during adaptive sampling. The contour indicates the score function, where the color intensity increases with the score, and high-score regions appear in

red. **a** Sort-and-select approach. **b** K-means clustering-based sampling approach: high-score candidates (left), clustering results with centroids (middle), and final selected samples (right)

cated by the small white dots in the left panel of Fig. 6b. Next, k-means clustering is performed to group adjacent samples, where the number of clusters is set equal to the number of new samples to be added. Finally, the centroids of the clusters are selected as new samples. This approach prevents the selected samples from being concentrated in a narrow region, thereby enhancing the diversity of new samples. This process is repeated until either the model achieves satisfactory accuracy or the computational budget is exhausted.

3 Training and validation of airfoil brain

In this section, the training and validation results of *Airfoil Brain* are presented. Two cases are considered. The first case evaluates the effectiveness of the proposed adaptive sampling method compared to conventional sampling approaches. A two-parameter problem, with varying Ma and α , is considered to facilitate intuitive visualization of the adaptive sampling behavior. The second case demonstrates the framework's applicability to the more challenging problem – C81 table prediction task. This case incorporates additional input variables, such as latent variables and flow conditions (Ma , Re , and α). The following subsections provide the detailed settings and results for each case.

3.1 Effectiveness of adaptive sampling: a two-parameter case study

3.1.1 Data preparation

To validate the effectiveness of the proposed adaptive sampling, a two-parameter benchmark problem was selected. This simplified setting allows for intuitive visualization of the sampling process in a two-dimensional input space, while facilitating repeated numerical experiments with manageable computational cost to obtain reliable results. The problem involves predicting the aerodynamic coefficient for varying Ma and α , with the airfoil geometry fixed to NACA23012.

In order to construct the database, the structured-grid CFD solver KFLOW was employed (Park and Kwon 2004; Hong et al. 2022). Since the C81 tables span a wide range of flow regimes — from steady to unsteady — both steady and unsteady RANS simulations were performed. Each case was initially computed under steady-state assumption. If the solution exhibited oscillatory residuals or divergence behavior, an unsteady simulation was sequentially conducted. Finally, the aerodynamic coefficients obtained from the unsteady simulations were time-averaged and incorporated into the dataset for training the surrogate model.

The Spalart-Allmaras turbulence model (Spalart and Allmaras 1992) was employed for turbulence closure. O-type grids were constructed with 405 and 81 points for chordwise

and normal direction, respectively. To get accurate resolution of wall boundary layer, the first grid spacings for normal direction were set to 10^{-6} in order to satisfy $y^+ \simeq 1$. The far-field boundary was positioned 40 chord lengths away from the aerodynamic center of the airfoil. For accurate and robust simulation, AUSMPW+ (Kim et al. 2001) upwind-biased flux function was utilized with second-order accurate total variation diminishing Koren limiter (Koren 1993). First and second-order accurate backward difference formula were used for steady and unsteady simulation, respectively, as the temporal integration scheme with diagonalized alternating direction implicit method. In the case of the unsteady simulation, time step was set as 0.005 times convective time unit (CTU). Dual time stepping was also used for robustness of the unsteady simulation. The simulations were conducted until the time reached 50 CTU.

The flow conditions, Ma and α were varied within the ranges $[0.2, 0.8]$ and $[-15^\circ, 25^\circ]$, respectively. The Re was adjusted proportionally to Ma , following the common practice for constructing C81 tables for airfoils (Kallstrom and Shirazi 2024). The Re corresponding to $Ma = 1.0$, Re_b was set at 5×10^6 .

We first generated multiple sets of training data using LHS, specifically 50, 100, 200, 500, and 1,000 points. Note that LHS was conducted separately to assess the impact of the number of training samples on the model. This process yielded a total of 1,850 data samples, serving as a comprehensive data pool.

To efficiently investigate the effect of adaptive sampling under various scenarios, this data pool was utilized as a source for additional samples. When the adaptive sampling algorithm requires additional sample points, it selects the closest samples from this data pool, rather than generating new samples from scratch, and it adds them to the training data for the next iteration.

For the test data, 533 samples were separately generated using full-factorial sampling with a 13-point grid for Ma and a 41-point grid for α . Figure 7 shows the distributions of training and test data in $Ma-\alpha$ domain. Training data points, generated by LHS, are shown in Fig. 7a, while test data points from full-factorial sampling are shown in Fig. 7b. For test data, we explicitly labeled the pre-stall and post-stall regions by identifying the local maximum and minimum points of lift coefficients through visual inspection. Since the flow characteristics vary significantly across stall conditions (or the boundaries between steady and unsteady flow regimes), these labeled data allow more detailed analysis across different flow regimes.

3.1.2 Training details

Before training the model, the database is first properly pre-processed. The current database includes Ma and α as input

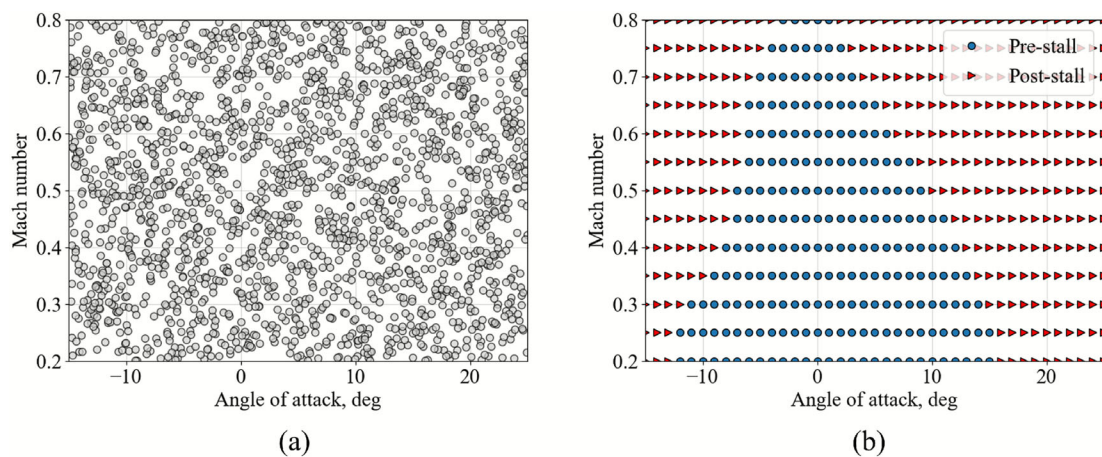


Fig. 7 Design of experiments for the two-parameter benchmark problem; **a** training data, **b** test data

parameters and the aerodynamic coefficients of lift, drag, and pitching moment as output. Among these, the drag coefficient has a significantly smaller variation compared with the lift and pitching moment coefficients. Since this differences in magnitude do not indicate importance, a logarithmic transformation is applied to the drag coefficient before training. Subsequently, both input and output data are standardized.

For the training settings, the Adam algorithm was selected as the optimizer. An initial learning rate of 10^{-3} , which decays by a factor of 0.5 every 5000 epochs, over a total of 50,000 epochs, together with an L_2 regularization coefficient of 10^{-5} was employed. These hyperparameters were chosen through preliminary runs to achieve stable convergence. A single batch is used for training, as it provided more stable convergence in this setup. Given the importance of proper hyperparameter selection for generalization performance of trained NN, hyperparameter tuning was performed using a grid search (see Appendix B.2).

3.1.3 Validation of adaptive sampling

This section aims to validate the effectiveness of the proposed adaptive sampling approach. To investigate the effect of balancing parameter α_w of score function, contours of model and data uncertainty, and the score function with respect to α_w is presented in Fig. 8. The white square symbol represents the training data, and the red triangle symbol represents the new samples selected based on the score function. Three cases are investigated: $\alpha_w = -1.0, 0.0, 1.0$, where each case represents total uncertainty (a simple sum of model and data uncertainty), model uncertainty, and model uncertainty penalized by data uncertainty.

In Fig. 8b, the model uncertainty effectively captures the sparsity of the data, showing high uncertainty in regions with sparse data. However, within the subsonic flow regime – specifically in the range of $Ma = [0.2, 0.8]$ and $\alpha =$

$[-15^\circ, 25^\circ]$ – the model uncertainty remains low, even in sparsely sampled regions. This lower uncertainty in the subsonic region is due to the linear nature of the subsonic flow, which the model can predict more accurately, thereby reducing model variability. This confirms that DE effectively quantifies model uncertainty and accurately identifies regions where model error is likely to be high, an essential capability for guiding adaptive sampling.

On the other hand, in Fig. 8c, the data uncertainty shows a different trend. High data uncertainty is observed in regions with large angles of attack, where the flow gradually becomes unsteady and more susceptible to numerical noise. Unlike the model uncertainty, the data uncertainty does not show a clear correlation with the data sparsity, as it is more influenced by the noise and unsteady characteristics inherent in these flow regimes, whose improvement through additional sampling is marginal beyond a certain level.

Figures 8d–f show the score functions defined by combining these uncertainties to guide adaptive sampling. In Fig. 8d, which shows the total uncertainty, we observe that adaptive sampling is clustered around regions with high negative angles of attack. This is because both data and model uncertainty are high in this region. However, this phenomenon may not be desirable, as additional sampling in high-noise regions may not significantly improve model accuracy. To address this, Fig. 8e considers only model uncertainty. Here, samples are more evenly distributed near stall boundaries, where model error is typically higher. Still, some clustering remains, similar to the case $\alpha_w = -1.0$. Figure 8f introduces a score function that penalizes regions with high data uncertainty, targeting areas likely to yield meaningful model improvements while avoiding noisy regions. This penalized criterion leads to a more focused distribution of new samples around stall boundaries compared to the $\alpha_w = 0.0$ case. From this analysis, we conclude that values of α_w between 0.0 and 1.0 are appropriate for adaptive sampling, and this

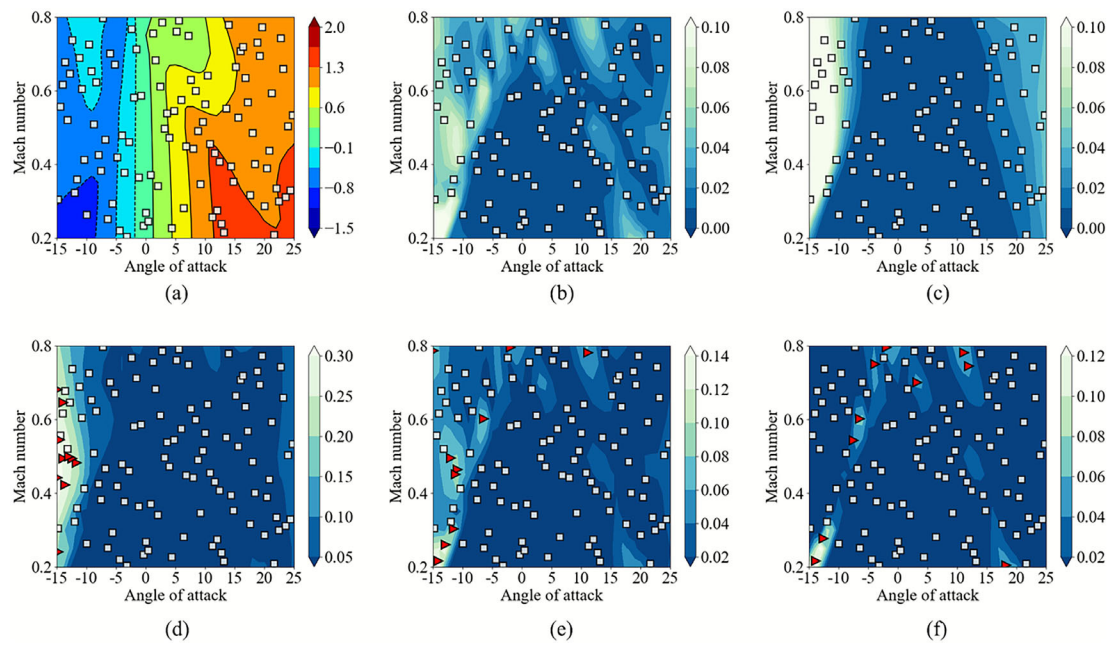


Fig. 8 Contours of predictive mean, uncertainty, and score function from Latin Hypercube Sampling (LHS) of 100 cases; **a** lift coefficient, **b** model uncertainty, **c** data uncertainty, **d** score at $\alpha_w = -1.0$, **e** score at $\alpha_w = 0.0$, **f** score at $\alpha_w = 1.0$

range was subsequently used for quantitative performance evaluation.

To investigate the effect of balancing parameters on the efficiency of adaptive sampling, numerical experiments were conducted with α_w values of 0.0, 0.2, 0.5, and 1.0. Starting with an initial training sample size of 100, we added ten new samples at each iteration over 20 iterations, resulting in a final total of 300 training samples. This sample size achieved an accuracy comparable to that of a model trained using LHS with a sample size between 500 and 1000, demonstrating that 300 samples provided a sufficient number of iterations for numerical experiments to observe the adaptive sampling trend effectively.

Figure 9 shows the convergence history of adaptive sampling for different values of α_w . The x-axis represents the number of training samples, while the y-axis indicates the improvement ratio, defined as the ratio of the mean absolute error (MAE) between the initial model and the adaptively updated model. Each colored line corresponds to the results for different α_w .

Since adaptive sampling involves inherent randomness, each experiment was repeated five times (dashed lines with reduced opacity), and the results were averaged (solid line). To assess the effect of adaptive sampling across different flow regimes, the results are separated into pre-stall and post-stall regions. For comparison, results obtained using LHS without adaptive sampling are also included as a reference (solid black line).

Figure 9 demonstrates that adaptive sampling achieves significantly higher accuracy than LHS with the same number of training samples. In most cases, adaptive sampling with approximately 200 samples outperforms LHS using 500 samples, highlighting its efficiency advantage.

The influence of α_w on model accuracy reveals a clear trend in the pre-stall region: higher values of α_w lead to greater accuracy. For instance, when $\alpha_w = 0.0$, the model shows minimal improvement, and its performance remains comparable to that of LHS sampling. As α_w increases, however, the accuracy progressively improves. For instance, with $\alpha_w = 1.0$, only 50 additional samples are sufficient to achieve the same accuracy as 500 LHS samples. This is because increasing α_w prioritizes highly nonlinear regions while penalizing noisy regions, thereby allocating new samples to areas where they most improve model accuracy.

Conversely, in the post-stall region, increasing α_w up to 0.5 has little effect on accuracy – a desirable outcome, as it enables additional samples to focus on the pre-stall region without degrading post-stall accuracy. This implies that post-stall regions are less sensitive to additional samples due to their highly noisy nature. This justifies penalizing these regions and prioritizing regions with strong nonlinearity. However, when $\alpha_w = 1.0$, the accuracy in the post-stall region deteriorates sharply, resulting in a substantial increase in total error. This degradation occurs because overly penalizing certain regions induces data imbalance, causing the loss function to underrepresent post-stall samples and leading the model to underfit these regions. These results suggest that

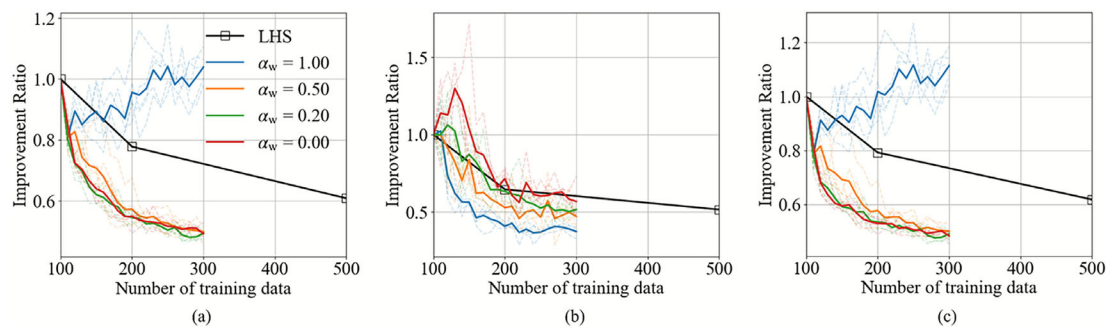


Fig. 9 Convergence history of adaptive sampling with respect to the number of training data. Results obtained by Latin hypercube sampling (LHS) and the proposed adaptive sampling with different weight parameters are compared; **a** total region, **b** pre-stall region, **c** post-stall region

a moderate α_w value between 0.2 and 0.5 should be used to yield optimal performance in both the pre- and post-stall regions.

These results highlight that the proposed score function effectively assigns weights across different flow regimes and successfully guides the adaptive sampling process. Notably, no external information is provided by the user regarding whether a region is nonlinear or noisy; the score function relies solely on the data and model uncertainty obtained from the DE model. This makes it particularly well-suited to multi-parameter problems, where identifying such nonlinear or noisy regions manually is difficult or even infeasible.

3.2 Prediction of C81 tables: a multi-parameter case study

3.2.1 Data preparation

To prepare the dataset, the CFD solver settings were identical to those described in Section 3.1.1. Given that the objective of this section is to predict C81 tables across a wide range of airfoil shape variations, both flow conditions and airfoil shapes are considered as input parameters. The ranges of these parameters are determined by considering the typical conditions for rotor and propeller designs. For flow conditions, Ma , α , and Re were varied in the range of $Ma = [0.1, 0.85]$, $\alpha = [-20^\circ, 25^\circ]$, and $Re_b = [2.0 \times 10^6, 6.0 \times 10^6]$, where Re_b is the reference Reynolds number at $Ma = 1.0$ that can be calculated as Re/Ma . For airfoil geometries, the latent variables of the AGN were adjusted to cover maximum thickness and camber ranging from 6% to 25% and 0% to 6% of the chord length, respectively. Since the physics latent variables of AGN are directly aligned with maximum thickness and camber, these ranges can be specified straightforwardly. The randomly sampled airfoil candidates from this design space, illustrated in Fig. 10, exhibit diverse geometric characteristics, including maximum thickness, maximum camber, leading edge radius, and trailing edge angle.

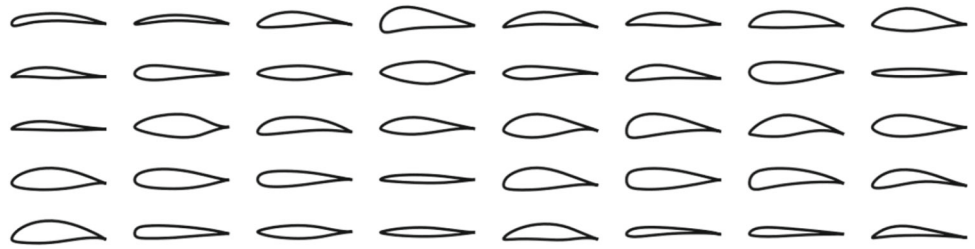
A total of 30,000 initial samples were generated using LHS. Of these, 1,500 samples were allocated for validation (hyperparameter tuning) and testing (performance evaluation), respectively. Subsequently, uncertainty-guided adaptive sampling was performed to progressively refine the model, by adding 1,000 samples in each iteration. The adaptive sampling process continued until the total number of model evaluations reached the predefined computational budget. In total, 96,750 samples were used for training the prediction model.

However, these test samples are drawn from an LHS initialization, with each sample corresponding to a unique combination of airfoil geometry and flow condition. As a result, samples for a specific airfoil are sparsely distributed, making it difficult to visualize continuous quantities such as lift–drag polars for individual airfoils. Therefore, for more detailed inspection, an additional test set was constructed using full-factorial sampling, based on representative airfoils such as Clark Y, NACA0012, NACA23012, RAE2822, RAE5214, and SC1094R8. The full-factorial sampling was conducted with the grid $Ma = \{0.4, 0.75\}$, $\alpha = \{-20^\circ, -19^\circ, \dots, 25^\circ\}$, and $Re_b = \{3.0 \times 10^6, 5.0 \times 10^6\}$. In summary, the original LHS test set is primarily used for quantitative analysis to evaluate global performance, assessing overall model accuracy across the entire parameter domain through statistical measures such as cumulative error distributions (Figs. 11 and 13). In contrast, the additional full-factorial test set is used for qualitative analysis of local performance, where detailed aerodynamic characteristics — such as lift–drag polars of representative airfoils — are examined to investigate model behavior for specific airfoil geometries (Figs. 12 and 14).

3.2.2 Validation of adaptive sampling

To investigate the effectiveness of the adaptive sampling method presented in Section 2.4, the errors for each aerodynamic coefficient are monitored during adaptive sampling iterations. Since the proposed adaptive sampling method is designed to reflect the local characteristics of response sur-

Fig. 10 Random airfoil geometries generated within the design space defined by the airfoil generation network



faces based on both data and model uncertainty, it is desirable to decompose the error with respect to different flow regimes. To this end, the test data are partitioned into four groups: transonic/steady, transonic/unsteady, subsonic/steady, and subsonic/unsteady. The flow regime is classified based on the Mach number and the unsteadiness of the aerodynamic coefficient. Specifically, transonic conditions are defined for $Ma \geq 0.65$, and subsonic for $Ma < 0.65$. A flow is classified as unsteady if the unsteadiness of the coefficient within a window of 20 CTUs exceeds 0.05%, where unsteadiness is quantified as $(\max - \min)/\text{mean}$.

Figure 11 illustrates the improvement ratio history for each region throughout the adaptive sampling process. For each case, the raw values are scattered as circular markers. We employed the Savitzky–Golay filter (Savitzky and Golay 1964), with a window length of 11 and a polynomial order of 2, to present the overall trend of improvement while preserving the local variations of the data.

It can be observed that the transonic/steady region is the most benefited region, followed by the subsonic/steady region. In contrast, unsteady regions were less responsive to adaptive sampling. This phenomenon can be anticipated from the definition of the score function in Eq. 7. Unsteady flow regimes tend to have noisier response surfaces, leading to higher data uncertainty. Therefore, these regions yield low score function values, penalized by the term $\alpha_w \sigma_{\text{data}}$. In contrast, transonic flow regimes exhibit more nonlinear behavior compared to subsonic regimes, resulting in higher model uncertainty and thus are encouraged by the term σ_{model} . As a result, the transonic and steady regions are the areas that benefit most from the proposed adaptive sampling approach. This trend is particularly beneficial, as steady and transonic regimes are closely associated with the dominant operating conditions of rotor blades. Although unsteady flows can play a critical role in specific scenarios, accurate modeling of steady/transonic behavior is often prioritized, given their direct relevance to the primary objectives of optimization tasks.

3.2.3 Validation of predictive mean

Through adaptive sampling, the PPN is efficiently trained under a limited computational budget. In this section, the trained PPN is validated using test samples that were not

included in the training process. The validation is conducted from both global and local perspectives to comprehensively evaluate the model's performance: the global validation (Table 1) assesses the overall predictive accuracy across the entire domain, whereas the local validation (Figs. 12 and 14) focuses on specific airfoil geometries, allowing for a more detailed inspection of prediction quality at individual design points.

For error metric, normalized means-absolute-error (NMAE) is used that is defined as follow:

$$\text{NMAE} = \frac{\|\mathbf{y}_{\text{CFD}} - \mathbf{y}_{\text{PPN}}\|}{\|\mathbf{y}_{\text{CFD}}\| + \epsilon} \quad (8)$$

where ϵ is the small value (ten times to the aerodynamic counts -10^{-3} for C_l , C_m , 10^{-4} for C_d) introduced to prevent numerical instability.

Table 1 summarizes the cumulative percentage of test samples whose NMAE falls within specified error bounds for C_l , C_d , C_m . The results indicate that approximately 30 % of the predictions exhibit errors below 1 %, and more than 70 % fall within 5 %, demonstrating the high accuracy and stability of the trained model across all aerodynamic coefficients. The full cumulative distributions of the aerodynamic coefficients are presented in Appendix C.2.

To qualitatively investigate the accuracy of the PPN for unseen airfoil shapes, the lift-drag polar of six representative airfoils was also plotted in Fig. 12. Both the polars obtained from CFD and the PPN are displayed for four different flow conditions: (i) $Re_b = 3 \times 10^6$, $Ma = 0.4$ (blue); (ii) $Re_b = 3 \times 10^6$, $Ma = 0.75$ (green); (iii) $Re_b = 5 \times 10^6$, $Ma = 0.4$ (red); and (iv) $Re_b = 5 \times 10^6$, $Ma = 0.75$ (orange). In subsonic conditions, the predictions match well with the CFD results. In transonic conditions, the results are comparable, although there are some over-predictions of the drag coefficient in the RAE airfoil series. Nevertheless, it can be confirmed that the overall trends influenced by variations in airfoil shape and flow conditions are accurately captured.

3.2.4 Validation of predictive uncertainty

While the PPN predicts aerodynamic coefficients with satisfactory accuracy, certain samples still exhibit high NMAE values exceeding 10%. These noticeable errors arise from

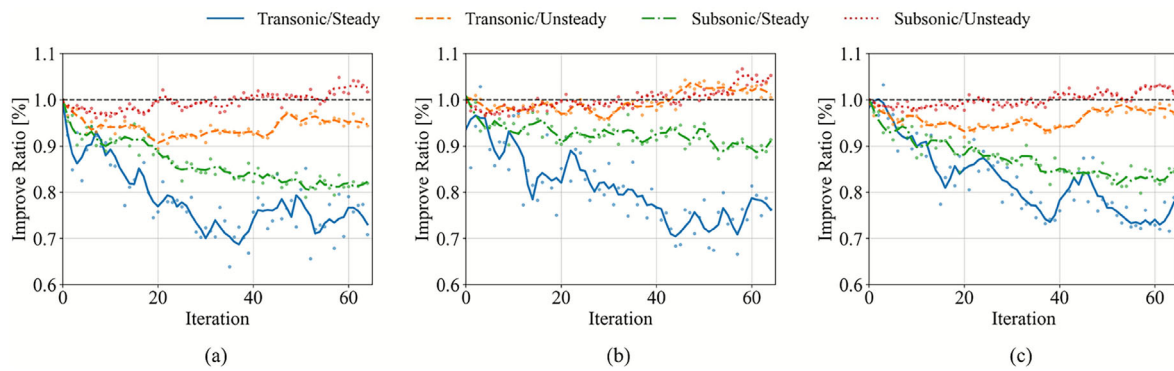


Fig. 11 Improvement ratio history of aerodynamic coefficients during the adaptive sampling process. Flow regimes are categorized into four groups based on Mach number and unsteadiness: transonic/steady,

transonic/unsteady, subsonic/steady, and subsonic/unsteady; **a** lift coefficient, **b** drag coefficient, **c** pitching moment coefficient

Table 1 Cumulative percentage values for different error bounds of NMAE

	C_l	C_d	C_m
NMAE $\leq 1\%$	29.4%	28.0%	20.5%
NMAE $\leq 5\%$	67.4%	71.0%	57.9%
NMAE $\leq 10\%$	80.0%	85.5%	73.1%

the wide range of flow regimes considered in our framework – particularly transonic and unsteady flows, where aerodynamic responses are highly sensitive to variations in airfoil shape. To ensure the PPN can be trusted in real applications, it is crucial that such model errors are well captured by predictive uncertainty. In this context, we proceed to validate the quality of the predictive uncertainty.

To measure the quality of predictive uncertainty, the area under the calibration error curve (AUCE) is used, a popular metric for uncertainty estimation (Yang and Yee 2024). AUCE measures the difference between the expected confidence interval (CI) and the observed CI. The expected CI is defined as the range of $\mu \pm a \cdot \sigma$, where a is a scale factor that determines the probability coverage of the CI. For example, in a Gaussian distribution, a 95% CI corresponds to $a = 1.96$, and a 99% CI corresponds to $a = 2.58$. The observed CI is computed as the fraction of test samples whose true values fall within the predicted confidence interval.

If the expected CI is larger than the observed CI, the trained model is considered under-confident (it estimates uncertainty higher than it should); conversely, if the expected CI is smaller, the model is over-confident (it estimates uncertainty lower than it should). To visualize AUCE, both the observed and expected CIs are plotted across a range of coverage levels from 0 to 1, as shown in Fig. 13. In the ideal case, the bars align with the ideal line $y = x$. As shown in

Fig. 13, the predicted confidence intervals closely match the observed values for all three quantities of interest, indicating that the DE model provides well-calibrated uncertainty estimates.

To further investigate the behavior of predictive uncertainty, we provide detailed validation results for a representative airfoil. Figure 14 presents the aerodynamic coefficient curves of the NACA23012 airfoil under subsonic ($Ma=0.4$, left column) and transonic ($Ma=0.75$, right column) flow conditions. The square symbols represent the aerodynamic coefficients obtained from CFD evaluations. The red and blue lines represent the predictive mean and uncertainty of *Airfoil Brain*, respectively. The red shaded region denotes the CI defined by $\mu \pm 3\sigma$ and MAE is plotted alongside as green bars.

Overall, the predicted aerodynamic curves (red lines) align well with the CFD results (squares), and the predictive uncertainty (blue line) varies depending on the flow conditions. Notably, predictive uncertainty spikes near the post-stall region, characterized by high positive or negative angles of attack. In contrast, regions with low angles of attack, where flow conditions typically remain steady, exhibit relatively low predictive uncertainty. Moreover, the predictive uncertainty closely matches with the MAE (green bar), confirming that the region with high model error is adequately represented as the high predictive uncertainty provided by DE.

These results demonstrate that the STD calibration method (Yang and Yee 2024) successfully calibrated the predictive uncertainty of the model. In this study, the uncertainty information was primarily used for adaptive sampling; however, accurate uncertainty estimation also enables informed decision-making in tasks such as Bayesian optimization and robust design optimization.

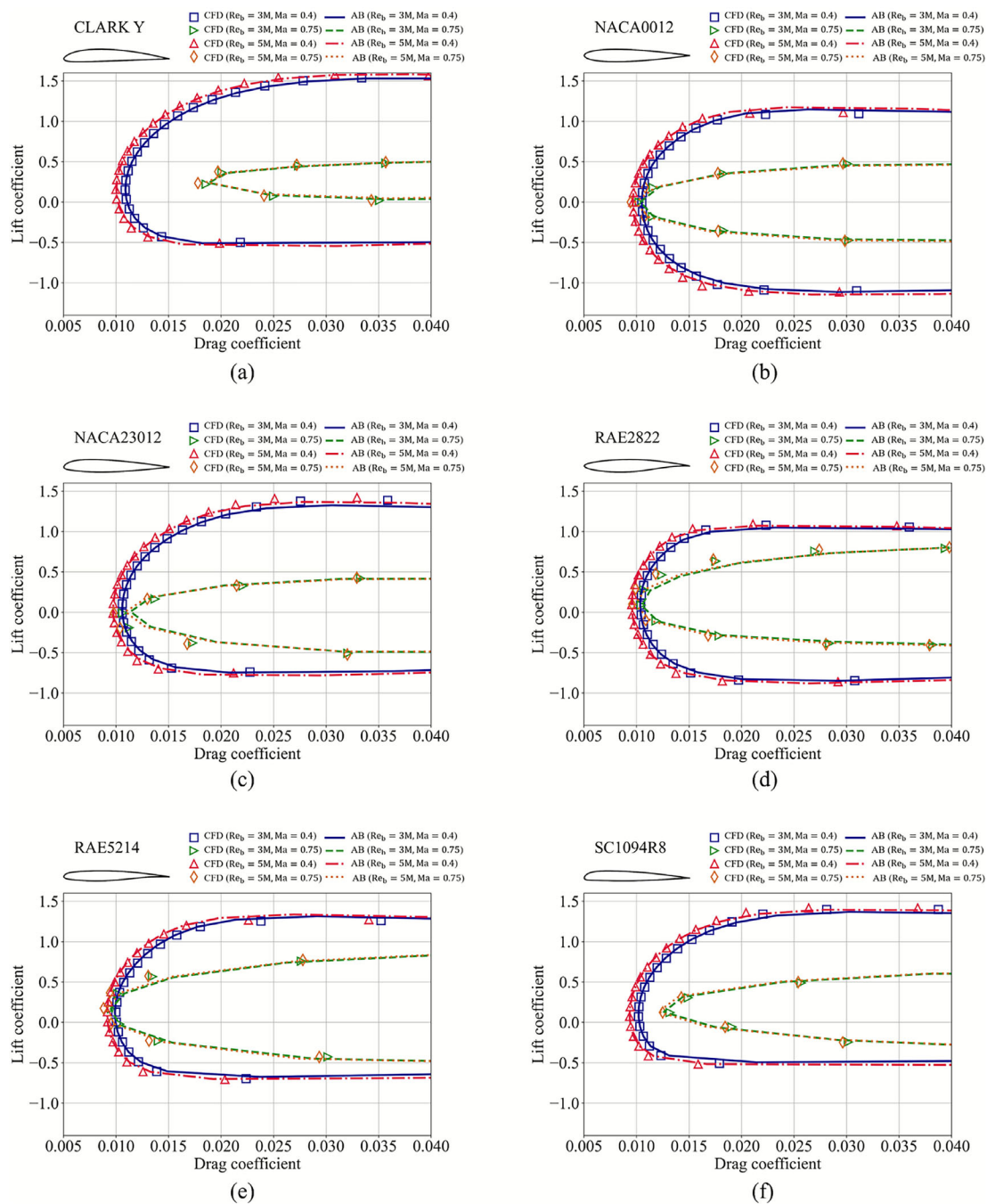


Fig. 12 Lift-drag polar curves of representative airfoils; **a** Clark Y, **b** NACA0012, **c** NACA23012, **d** RAE2822, **e** RAE5214, **f** SC1094R8

4 Application to multi-objective optimization of a propotor

4.1 Problem definition

The previous sections confirm that *Airfoil Brain* efficiently predicts C81 tables across a wide range of flow conditions and airfoil geometries in real time. This capability has significant potential for accelerating the blade optimization

process. Figure 15 presents a schematic illustration of the blade optimization process. In this framework, *Airfoil Brain*—specifically the PPN component—replaces high-fidelity CFD simulations by rapidly generating the C81 tables required for rotor blade performance evaluation. This capability enables on-demand generation of C81 tables throughout the design process, allowing airfoil shape parameters to be included as design variables and thereby expanding design flexibility in blade optimization.

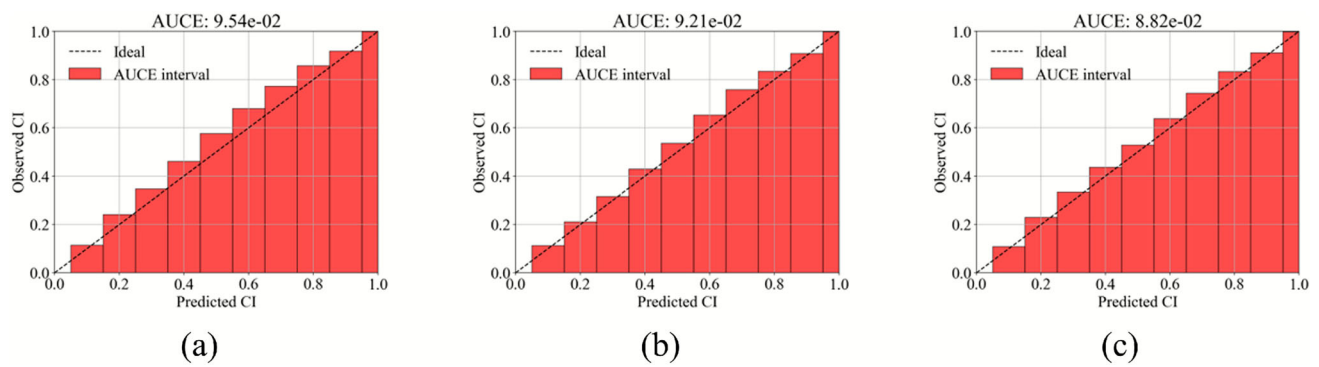


Fig. 13 Validation of predictive uncertainty of the *Airfoil Brain* model using area under the confidence-error curve (AUCE); **a** AUCE of C_l , **b** AUCE of C_d , **c** AUCE of C_m

Specifically, in conventional approaches using pre-computed C81 tables, each airfoil is treated independently, and sampling is performed densely only along the flow condition axis but sparsely along the geometry axis. Consequently, interpolation (or extrapolation) is limited to flow conditions, making and geometry-wise interpolation is inefficient and poorly scalable. In contrast, within the *Airfoil Brain* framework, both airfoil geometry and flow condition are considered simultaneously as input parameters during the sampling stage. This formulation greatly improves sampling efficiency and enables generalized aerodynamic predictions across a wide range of airfoil shapes.

To demonstrate the practicality of *Airfoil Brain*, an aerodynamic shape optimization of a proprotor was conducted. A proprotor was selected as the test case due to its challenging design requirements: the blades must operate under multiple flight conditions, namely hover and cruise. Under these conditions, each blade section encounters different flow regimes, making it essential to adapt blade geometry to local flow conditions in order to improve efficiency (Leishman and Rosen 2011). Since a well-designed proprotor must achieve high efficiency in both hover and cruise modes, a multi-objective optimization was performed.

The optimization problem in this study was formulated with reference to the XV-15 tilt-rotor aircraft (Maisel 2000), while the geometric and operating parameters were simplified. The detailed specifications of the baseline geometry and the operating conditions for this study are summarized in Tables 2–3. Note that the baseline blade section was set to NACA 0012, a symmetric and widely used reference airfoil, to provide a neutral aerodynamic reference for assessing the effects of airfoil shape variations in the optimized blade geometries. The aircraft weights was assumed to be 80,000 N, corresponding to a required thrust of 40,000 N per rotor in hover. The required thrust in cruise was approximated as one-fifth of the hover thrust, consistent with the

hover-to-cruise thrust ratio typically observed in tilt-rotor configurations (Droandi and Gibertini 2015).

For the aerodynamic analysis, we used BEMT, a widely used method for preliminary design owing to its cost efficiency (Ning 2014). The C81 tables required for BEMT analysis are generated using the *Airfoil Brain*. Since *Airfoil Brain* can quickly evaluate the C81 tables with varying airfoil geometry, both planform and blade sections can be incorporated as design variables for optimization. For the planform, chord and twist distributions along the radial direction are parameterized using B-splines to ensure smooth variations, as illustrated in Fig. 16. The number of controllable parameters was set to six for chord (Fig. 16a) and five for twist (Fig. 16b), resulting in a total of eleven planform parameters. Examples of the generated planform samples using these parameterizations are illustrated as lightly faded curves in Fig. 16. For the blade sections, four airfoils are used to define the sectional distribution. As shown in Fig. 17, the locations of two airfoils are varied, while the others are fixed at $r/R = 0.2$ and $r/R = 1.0$. All four airfoil shapes were varied, each parameterized by ten latent variables using an AGN. The blade sections between the four airfoils were generated by linear interpolation within the latent space, which provides a smoother transition between different airfoil shapes. In summary, the total number of design variables for planform and airfoil sections is 11 and 42, respectively.

The objective functions are power coefficients (C_P) in hover and cruise modes, where $C_P = P/\rho A(\Omega R)^3$. Constraints are applied to the thrusts in hover and cruise modes to ensure that the proprotor generates sufficient lift to overcome the aircraft weight and drag, respectively. In addition, constraints are imposed on the maximum thickness and thrust-weighted solidity to ensure structural integrity. The blade optimization problem can be formulated as follows:

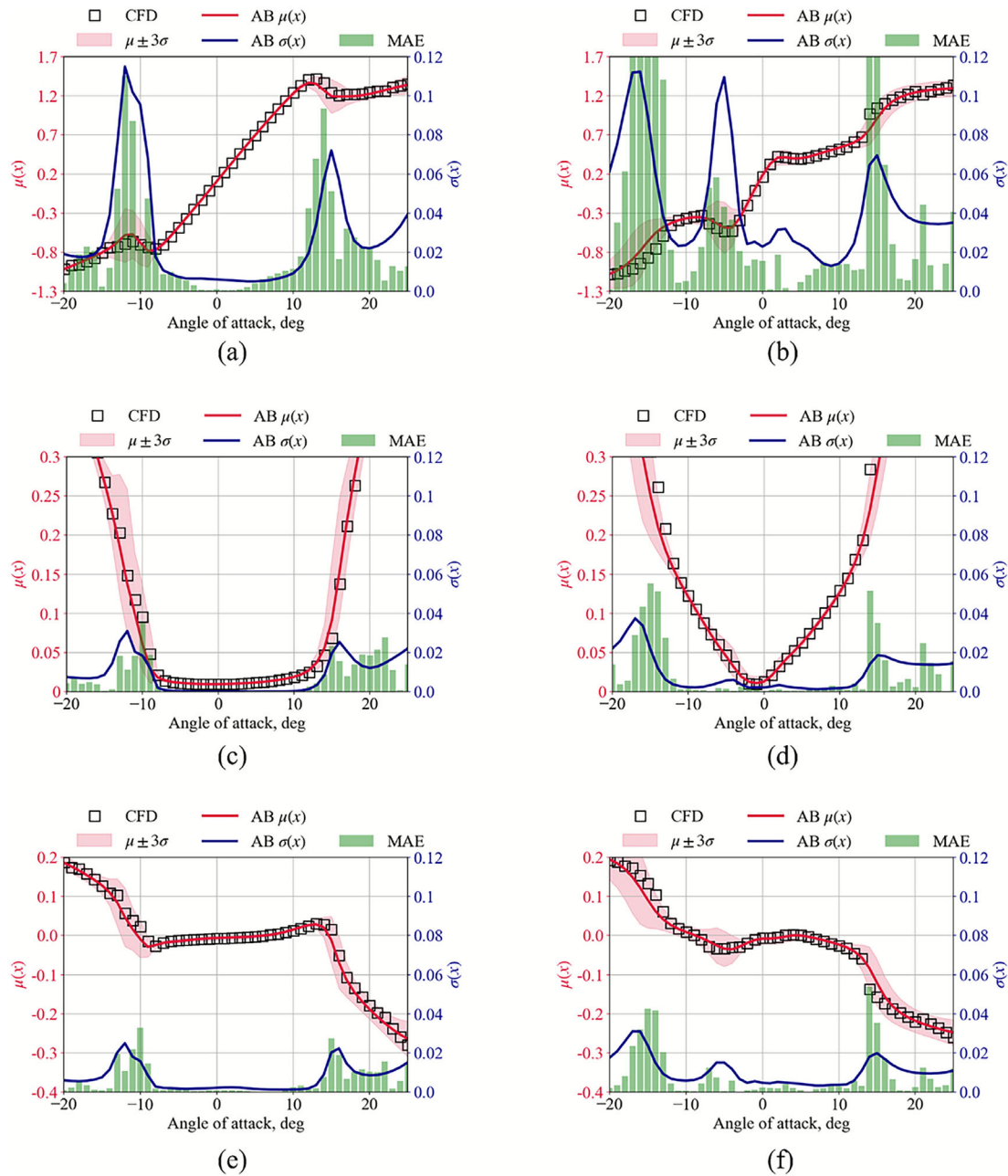


Fig. 14 Predicted aerodynamic coefficients of the NACA 23012 airfoil under subsonic ($Ma = 0.4$) and transonic ($Ma = 0.75$) conditions at $Re_b = 5.0 \times 10^6$; **a** C_l at subsonic, **b** C_l at transonic, **c** C_d at subsonic, **d** C_d at transonic, **e** C_m at subsonic, **f** C_m at transonic

Objective:

$$\min_{\mathbf{x}=\{\mathbf{DV}_{\text{planform}}, \mathbf{DV}_{\text{airfoil}}\}} \mathbf{F}(\mathbf{x}) = [C_{P,\text{hover}}(\mathbf{x}), C_{P,\text{cruise}}(\mathbf{x})],$$

subject to:

$$\begin{cases} T_{\text{hover}}(\mathbf{x}) \geq T_{\text{req,hover}}, \\ T_{\text{cruise}}(\mathbf{x}) \geq T_{\text{req,cruise}}, \end{cases} \quad (\text{Thrust constraints})$$

$$t_{\text{max},i}(\mathbf{x}) \geq t_{\text{max,baseline}}, \quad i = 1, \dots, 4, \quad (\text{Thickness constraints})$$

$$\sigma_w(\mathbf{x}) \geq \sigma_{w,\text{baseline}}, \quad (\text{Solidity constraint})$$

where the detailed constraint values are set to $T_{\text{req,hover}} = 40,000$ N, $T_{\text{req,cruise}} = 8,000$ N, $t_{\text{max,baseline}} = 12\%$, and $\sigma_{w,\text{baseline}} = 0.089$ corresponding to the required thrusts, the maximum thickness ratio, and the solidity, respectively. The design variable vector $\mathbf{x}$ consists of planform parameters $\mathbf{DV}_{\text{planform}}$, airfoil parameters $\mathbf{DV}_{\text{airfoil}}$, defined as follows:

Fig. 15 Schematic illustration of blade optimization using a genetic algorithm. High-fidelity CFD simulations are replaced with *Airfoil Brain*, enabling rapid prediction of C81 tables needed for rotor blade performance evaluation. This allows airfoil shapes to be considered as design variables, resulting in the concurrent optimization of airfoil and planform parameters

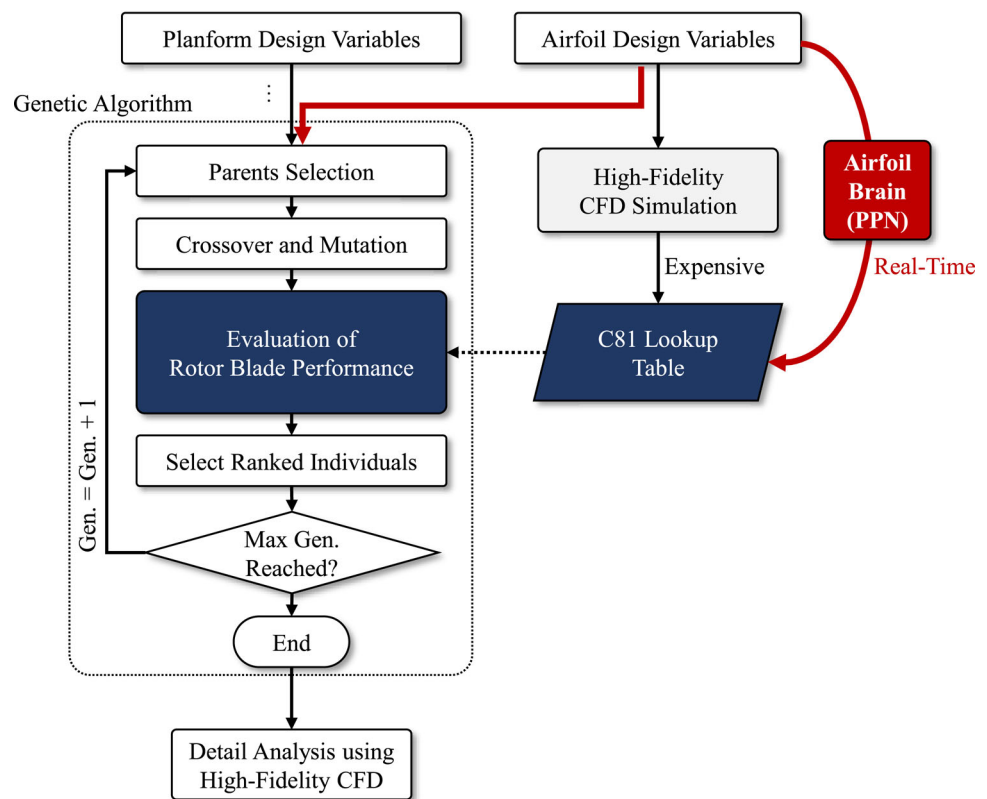


Table 2 Geometric specification of the baseline propotor

Parameter	Value
Number of blades	3
Blade radius (m)	3.81
Chord length (m)	0.3557
Taper ratio	1.0
Linear twist	−25°
Cross-section airfoil	NACA 0012

Table 3 Operating conditions of the propotor

Parameter	Hover mode	Cruise mode
Rotor speed (rad/s), Ω	61.6	48.1
Altitude (m), h	0	6,000
Tip Mach number, M_{tip}	0.691	0.580
Axial speed (m/s), V_{axial}	0	150
Required thrust (N), T_{req}	40,000	8,000

$$\begin{aligned}
 \mathbf{DV}_{\text{planform}} &= \underbrace{\{DV_{c1}, \dots, DV_{c6}\}}_{\text{Chord distribution}} \cup \underbrace{\{DV_{t1}, \dots, DV_{t5}\}}_{\text{Twist distribution}}, \\
 \mathbf{DV}_{\text{airfoil}} &= \underbrace{\{LV_{i,j} \mid i = 1, \dots, 4; j = 1, \dots, 10\}}_{\text{Latent variables for 4 airfoil sections}} \cup \\
 &\quad \underbrace{\{DV_{l1}, DV_{l2}\}}_{\text{Sectional locations}},
 \end{aligned}$$

Two optimization cases were considered: planform optimization with a fixed airfoil and concurrent optimization of planform and airfoil. For brevity, these are hereafter referred to as the *planform only* and *concurrent* optimization cases. In the planform only case, the design space is limited to the planform parameters (11 variables), whereas the concurrent case expands the design space to include both planform and airfoil shape parameters (53 variables). This compar-

ative analysis investigates the benefits of incorporating airfoil shapes as design variables, underscoring the practicality of the *Airfoil Brain*.

While the *Airfoil Brain* framework itself is differentiable and can be incorporated into a gradient-based optimization framework—which is indeed efficient in high-dimensional continuous spaces—the overall optimization framework often involves external aerodynamic solvers or performance analysis modules that are non-differentiable, limiting the use of gradient-based optimization. To ensure general applicability and solver independence, a population-based global optimizer was adopted, specifically the NSGA II genetic algorithm (Deb et al. 2000), which is well known for its effectiveness in handling multi-objective problems with constraints. The population size and number of generations were set to 100 and 300, respectively, which yielded sta-

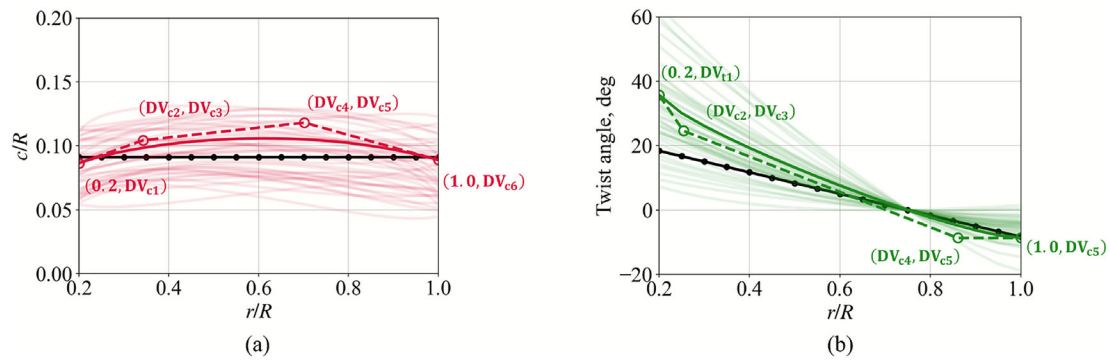


Fig. 16 B-spline parameterization of planform shape along the radial direction; **a** Chord length parameterization, **b** Twist angle parameterization

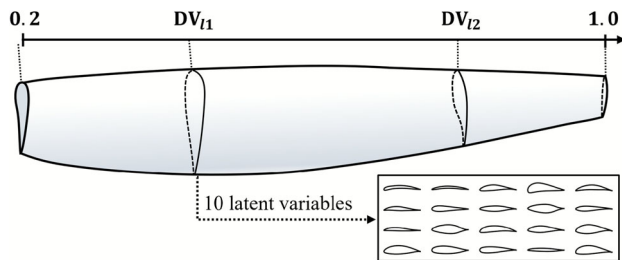


Fig. 17 Parameterization of blade sections was performed using four spanwise sections, where the airfoil shapes were represented by the airfoil generation network. Among these, two sections were fixed at the root and tip, while the spanwise locations of the remaining two sections are parameterized and denoted as DV11 and DV12

ble convergence and adequate search diversity in preliminary optimization trials.

4.2 Optimization results

The optimization results are presented in Fig. 18. The Pareto fronts of both optimization cases are shown, with blue and green markers representing the planform only and concurrent cases, respectively. The results show that the Pareto front of the concurrent case is superior to that of the planform only case. For extreme cases of hover and cruise modes, the required power coefficients are reduced by 4.00% and 2.23%, respectively. These results underscore the importance of concurrent optimization of both planform and blade sections.

To further investigate the optimization results, representative cases from each Pareto front were selected, as indicated by star-shaped markers in Fig. 18. These cases correspond to solutions with equal objective weights ($\alpha = 0.5$), and their geometries are visualized in Fig. 19. Detailed configurations along the radial direction are shown in Figs. 20 and 21.

For planform distributions, a similar trend is observed in both the planform only and concurrent cases in Fig. 20. First, there is a tendency to minimize the chord at the tip of the rotor. Second, the twist angles in the inboard sections increase, while those in the outboard sections decrease. However, the

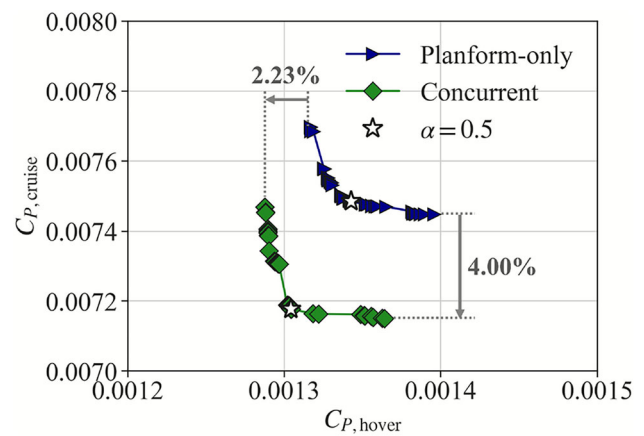


Fig. 18 Comparison of Pareto fronts of optimization cases. The baseline case is not included as it violates the design constraints

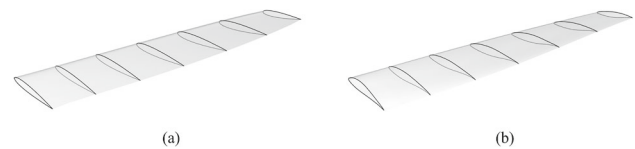


Fig. 19 Three-dimensional geometry of optimized propotor blades. **a** optimization considering planform only. **b** optimization considering both planform and airfoil shape

extent of these changes is more pronounced in the concurrent case, suggesting that the increased design flexibility due to the additional consideration of varying airfoil shapes along the radial direction also influences the planform design.

For the blade sections, the concurrent case demonstrates significant variation along the radial direction in Fig. 21. In the inboard section, the airfoil have a large leading edge radius and high maximum camber ($r/R = 0.20, 0.33$), which are beneficial for generating high lift in low-speed flow conditions. Conversely, in the outboard section ($r/R = 0.87, 1.00$), the airfoil is designed with a relatively small leading edge radius and low maximum camber, optimizing it for minimizing drag and reducing shock under high-speed flow conditions. In summary, increasing design flexibility

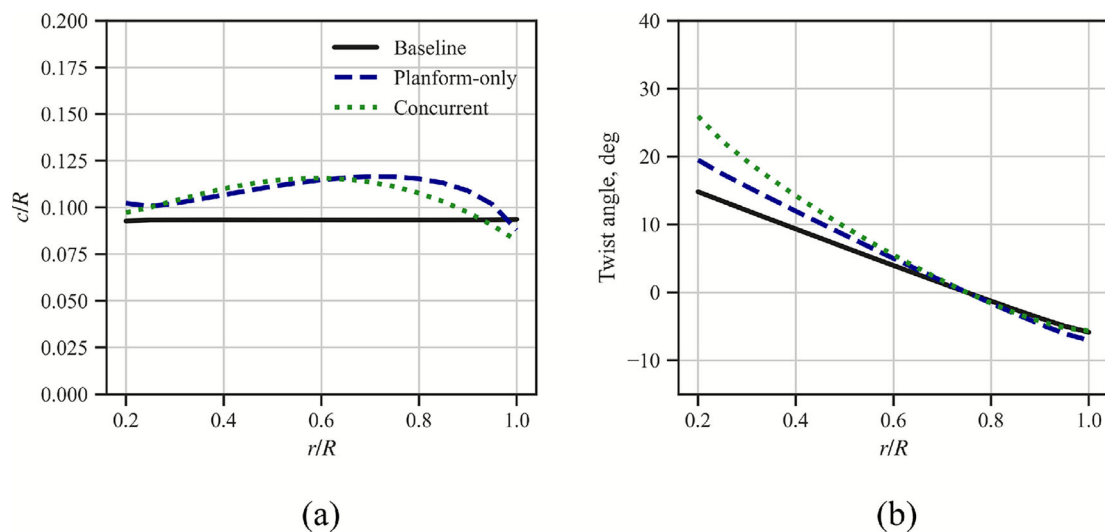


Fig. 20 Planform distributions of optimized propotor; **a** Chord length distributions, **b** Twist angle distributions

not only affects the planform configuration but also leads to significant variations in the airfoil sections along the spanwise direction, making them suitable for various local flow conditions. Such variations cannot be achieved when airfoil shapes are fixed or only partially varied, as in many conventional optimization approaches. This clearly highlights the benefit of the concurrent design strategy.

Since BEMT is a low-fidelity aerodynamic solver that does not capture complex three-dimensional flow phenomena such as tip vortex interaction, radial inflow, and wake contraction, it is important to verify the validity of the observed design trends. For this purpose, three-dimensional RANS simulations were performed on the optimized blades using the KFLOW solver. The simulations employed the moving reference frame technique and the SA-noft2 turbulence model (Aupoix and Spalart 2003). For highly accurate and robust solutions, a 5–6th-order accurate eMLP-VC reconstruction scheme (Hong et al. 2022, 2023) with the AUSMPW+ upwind-biased flux function (Kim et al. 2001) was utilized. The operating conditions are the same as those presented in Table 3, while the collective pitch angles are swept within the range $\theta_{c, \text{hover}} = [6^\circ, 18^\circ]$ and $\theta_{c, \text{cruise}} = [45^\circ, 55^\circ]$ to cover both on-design and off-design conditions. The details of the computational grid are presented in Appendix D.

Figure 22 presents the simulation results for both hover and cruise modes. The figure of merit (FM) and propulsive efficiency (η) are plotted on the y-axis against the thrust coefficient on the x-axis. Three blade configurations — baseline, planform only, and concurrent — are represented by black, blue, and green lines, respectively. The results obtained from both BEMT and CFD are overlaid for a direct comparison of the design trends, where the BEMT results are represented by dashed lines and the CFD results by solid lines. The on-

design condition, where the optimization was performed, is also indicated by a dotted gray line.

As can be seen, the concurrent case outperforms the other cases in both hover and cruise modes: it has higher FM and η than the planform only case. This trend can be also interpreted from the pressure coefficient contours on the blade surface. Figure 23 shows the pressure coefficient on the upper surface of the optimized propotor, with the surface pressure distribution near the tip ($r/R = 0.9$) visualized on the right. Here, the cases closest to the on-design conditions are shown. A strong shock is observed near the tip in the planform only case, while this shock is effectively eliminated in the concurrent case, resulting in smoother pressure distribution and improved aerodynamic efficiency.

Furthermore, even under off-design conditions (Fig. 22), the concurrent case maintains superior performance, particularly at higher thrust coefficients. These results are consistent with the theoretical insights from Ref (Leishman and Rosen 2011), which emphasize that tailored blade section design can broaden the operational envelope. This finding highlights the importance of concurrent optimization of airfoil shape and planform — an approach enabled by the *Airfoil Brain* framework.

Nevertheless, a noticeable difference exists between the BEMT and CFD results. Under hover condition, BEMT consistently underestimates the FM by approximately 0.05 ($\approx 6.25\%$ deviation), primarily due to its inability to capture complex three-dimensional flow phenomena. In contrast, this deviation is alleviated under the cruise condition, which may be attributed to the rapid downstream convection of tip vortices by the axial flow, thereby reducing their influence on the inflow.

Despite this discrepancy in absolute values, the relative efficiency among the baseline, planform only, and concurrent

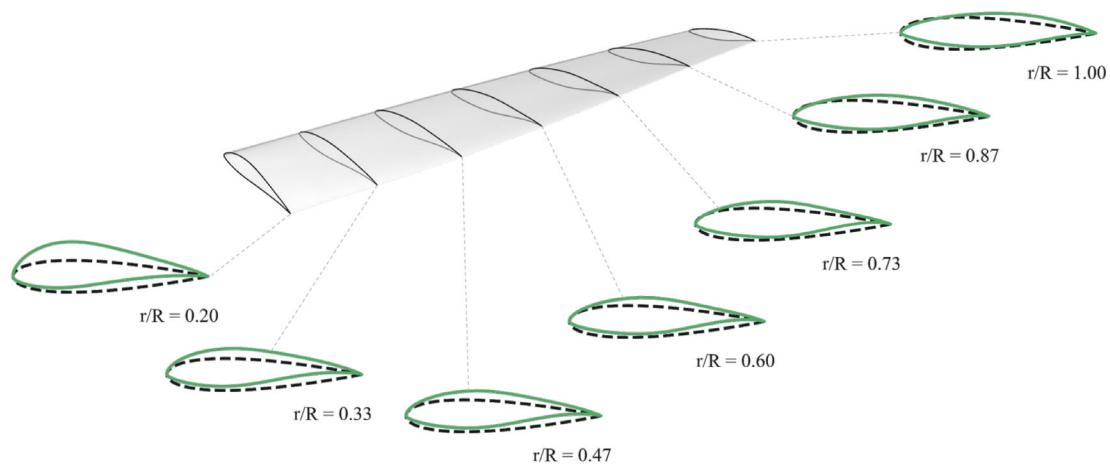


Fig. 21 Airfoil distributions along the blade of the optimized prop rotor. The black dotted line represents the baseline airfoil, while the green solid line indicates the optimized airfoil for the concurrent case

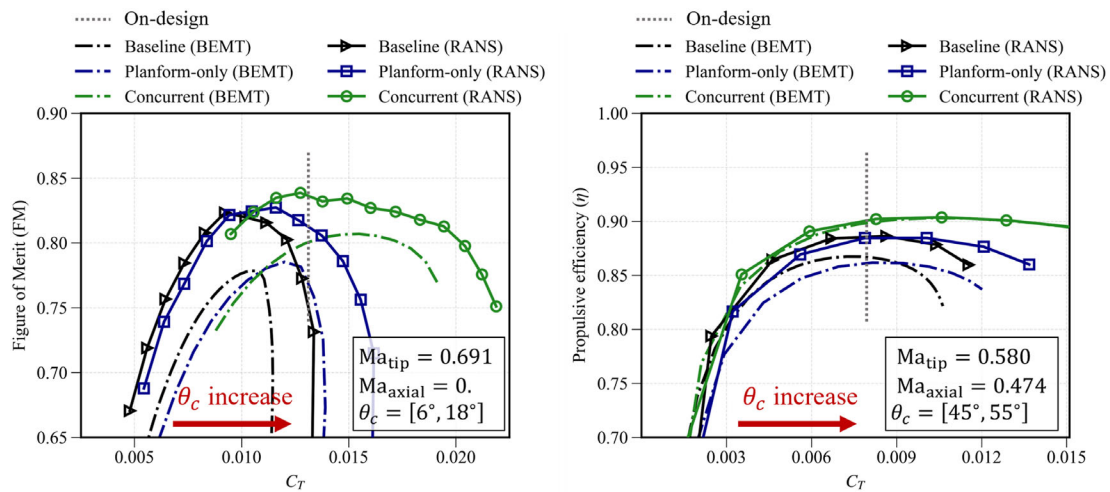


Fig. 22 Performance evaluation of representative prop rotor configurations based on BEMT and RANS simulations; **a** Figure of merit (FM) in hover mode, **b** Propulsive efficiency (η) in cruise mode

cases shows close agreement between the BEMT and CFD results. This consistency arises because the two-dimensional airfoil data used in BEMT already incorporate transonic and compressibility effects, allowing the low-fidelity model to reproduce the relative performance variations predicted by CFD. Therefore, the proposed framework retains a sufficient level of reliability for design studies, allowing designers, with careful consideration, to perform meaningful trend analyses at a fraction of the computational cost of CFD.

Furthermore, while demonstrated for a single configuration in this study, the framework can readily adapt to changes in design requirements — such as target thrust, flight speed, or geometric constraints — enabling rapid exploration of the rotor blade design space without the need for additional retraining of the surrogate model. This capability highlights its strong potential for applications in next-generation rotorcraft, urban air mobility vehicles, and wind energy systems.

5 Conclusion

This study proposes an efficient data-driven surrogate modeling framework, named *Airfoil Brain*, for real-time prediction of C81 tables to accelerate the blade optimization process. The framework addresses two major challenges arising from the need to cover diverse flow regimes across a wide range of airfoil geometries: (1) achieving compact and physically interpretable representation of airfoil shapes and (2) mitigating the computational cost of constructing large databases spanning multiple flow regimes. To address these, the framework integrates two core components: the Airfoil Generation Network (AGN) and the Performance Prediction Network (PPN). The AGN provides an efficient airfoil parameterization using compact and intuitive latent variables that ensures geometric feasibility and enables efficient exploration of the design space, while the PPN enables scalable and

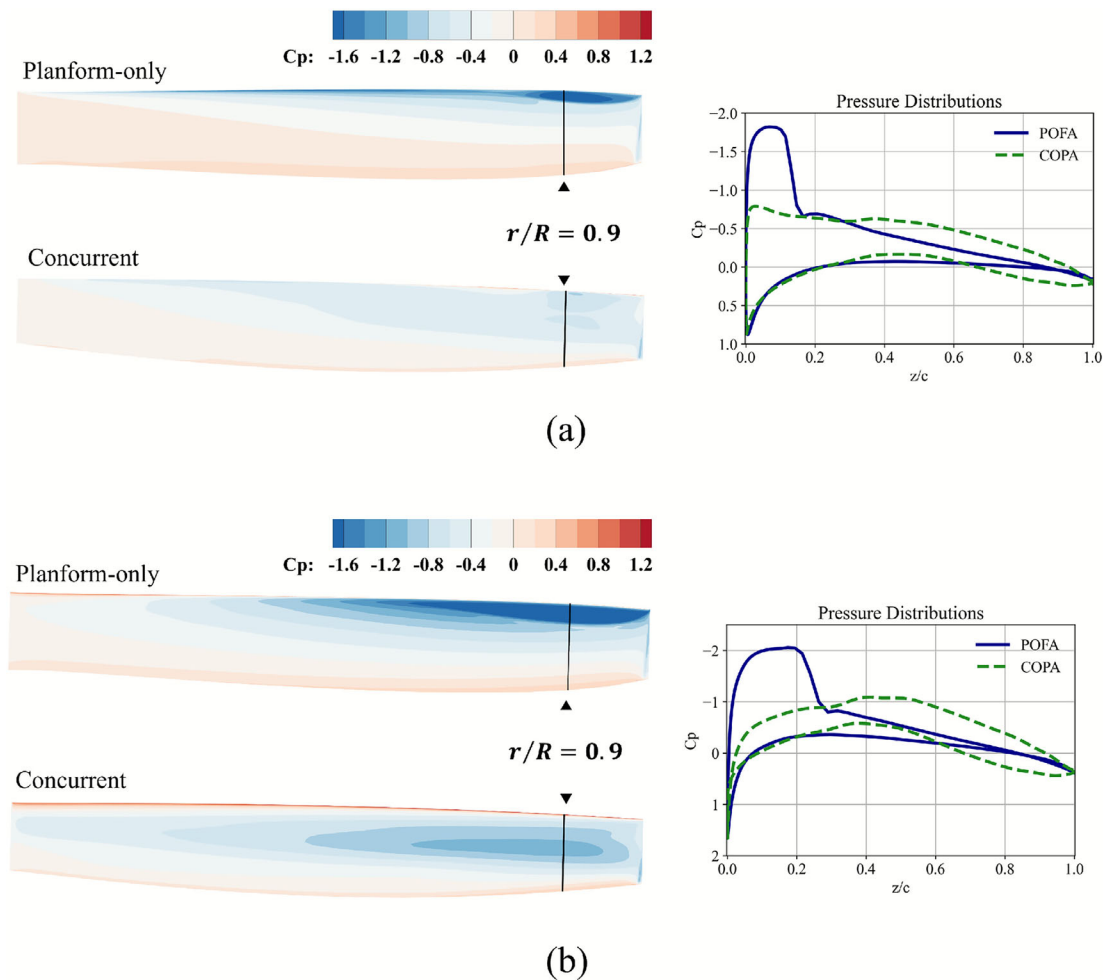


Fig. 23 Pressure coefficient (C_p) distribution on the upper surface of the optimized proprotor blade. **a** Hover mode. **b** Cruise mode. The right sub-panel in each mode shows the sectional pressure distribution at the radial location $r/R = 0.9$

uncertainty-aware surrogate modeling. The quantified predictive uncertainty further guides adaptive sampling, allowing computational resources to be concentrated in regions exhibiting complex aerodynamic behavior.

The efficiency of the adaptive sampling strategy over conventional methods, as well as the accuracy of the PPN, is validated using high-fidelity CFD simulation data. With *Airfoil Brain*, C81 tables can be generated in real-time during the blade optimization process, enabling airfoil shape parameters to be treated as design variables and thereby enhancing the design flexibility of rotor blades. To validate this capability, a multi-objective optimization of a proprotor was conducted to minimize the required power for both hover and cruise conditions. The results demonstrate that the increased design flexibility enabled by *Airfoil Brain* leads to improved aerodynamic performance. Three-dimensional RANS simulations further confirmed that the observed design trends remain valid under high-fidelity evaluations.

However, since the present optimization was performed using a low-fidelity aerodynamic solver, BEMT, it does not account for full three-dimensional effects — such as tip vortex interactions — in the optimization results. In addition, while the adaptive sampling strategy maximized data efficiency under a limited computational budget, this trade-off slightly reduced accuracy in unsteady flow regimes.

For future work, several areas deserve further exploration. One promising direction is to incorporate optimization under uncertainty, leveraging the PPN's capability to quantify predictive uncertainty. Integrating this information into Bayesian or robust optimization frameworks would enable more reliable decision-making during the design process. Another promising direction is to expand the surrogate model to predict field-level aerodynamic data (e.g., pressure distribution or boundary-layer parameters), and, further, to incorporate time-dependent flow solutions. Such extensions would allow the model to capture unsteady flow phenomena more accurately and facilitate integration with aeroacoustic

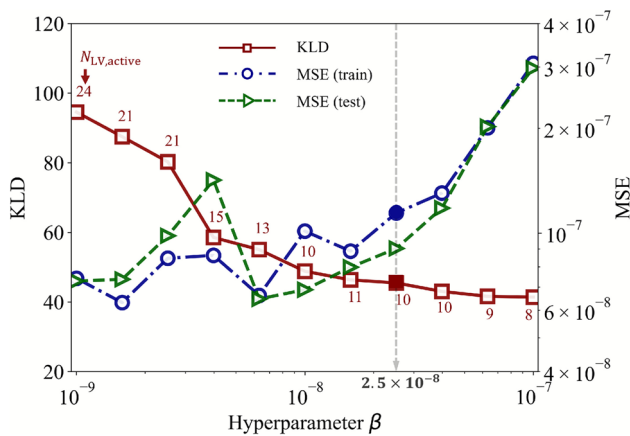


Fig. 24 Mean-squared-error (MSE) and Kullback–Leibler divergence with respect to the hyperparameter β

and structural analyses. These advances would contribute to a more comprehensive and reliable framework for aerodynamic performance prediction and rotor blade optimization.

Appendix A computational cost analysis

The total computational cost associated with data generation and model training is summarized in Table 4. The CFD simulations were performed on Intel Xeon Gold-6132 (28 cores). Among total of 96,750 CFD samples, approximately 36.5 % were steady-state simulations and 63.5% were unsteady simulations. The average computational cost per case was 0.23 CPU hours for steady and 3.2 CPU hours for unsteady simulations, resulting in an overall cost of approximately 200.6k CPU hours. In neural network training, the Airfoil Generation Network (AGN) and the Performance Prediction Network (PPN) required approximately 57 and 46 min, respectively, using an NVIDIA RTX 3090 Ti GPU.

For reference, generating a single airfoil's C81 lookup table — covering a parametric grid of 10 Mach numbers, 3 Reynolds numbers, and 46 angles of attack (1,380 points in total) — requires about 2.9 k CPU hours using CFD simulation. In contrast, the proposed framework produces the same

table within 10 ms of inference. Hence, although the computational time for data generation appears large, it represents a moderate investment, as the initial cost is outweighed by the computational savings in subsequent optimizations that require thousands of design evaluations.

Appendix B hyperparameter tuning results

B.1 Airfoil generation network

Hyperparameter tuning was conducted through a grid search procedure. The investigated hyperparameters included the sizes of filter and kernel, the dimension of latent space, activation function, and regularization parameter β . To alleviate the computational burden, the grid search was performed in two sequential stages. In the first stage, β was fixed to zero while varying the other parameters. Once the optimal configuration of these parameters was identified, a second grid search was conducted for β . The first hyperparameter tuning results are summarized in Table 5.

Figure 24 presents the results of hyperparameter tuning for β . The x-axis denotes β , while the y-axis shows the KL divergence and MSE. It is worth noting that in β -VAE, several latent dimensions are automatically deactivated by the pressure to minimize the KL divergence. To quantify this effect, each latent dimension was individually varied while fixing the others, and its influence was evaluated by the average standard variation of the airfoil y coordinates. The number of active latent variables, ($N_{LV, active}$), was defined as those with a mean standard deviation above 10^{-5} ; the corresponding values are annotated next to the red solid line in Fig. 24. It can be observed that ($N_{LV, active}$) gradually decreases with KL divergence.

From the hyperparameter tuning results, a clear trade-off can be observed: as the KL divergence decreases, the MSE increases. The MSE remains nearly constant up to $\beta = 2.5 \times 10^{-8}$, beyond which it rises exponentially. Meanwhile, the KL divergence decreases steadily with increasing β , but the rate of improvement diminishes beyond this point. Therefore,

Table 4 Summary of computational cost for data generation and model training

Process	Hardware	Processing time
Data generation (CFD)	Xeon Gold 6132	~200,600 CPU hours
Model training		
– Airfoil Generation Network (AGN)	RTX 3090 Ti	57 min
– Performance Prediction Network (PPN)	RTX 3090 Ti	46 min
C81 table generation (1,380 points)		
– CFD-based	Xeon Gold 6132	~2,900 CPU hours
– Model-based	RTX 3090 Ti	< 0.01 s

Table 5 Hyperparameter tuning results of AGN

Activation	Filter size	Latent dimension	Kernel size	Training loss
GELU	32	6	3	4.4×10^{-8}
			7	4.3×10^{-8}
		12	3	3.0×10^{-8}
			7	2.8×10^{-8}
	64	6	3	1.1×10^{-7}
			7	1.4×10^{-7}
		12	3	1.6×10^{-5}
			7	1.0×10^{-7}
	128	6	3	1.2×10^{-5}
			7	1.0×10^{-5}
		12	3	1.3×10^{-5}
			7	1.3×10^{-5}
LeakyReLU	32	6	3	4.4×10^{-8}
			7	4.4×10^{-8}
		12	3	3.2×10^{-8}
			7	3.0×10^{-8}
	64	6	3	1.9×10^{-7}
			7	2.0×10^{-7}
		12	3	2.1×10^{-7}
			7	2.8×10^{-8}
	128	6	3	1.3×10^{-5}
			7	1.1×10^{-5}
		12	3	2.2×10^{-5}
			7	1.0×10^{-5}
ReLU	32	6	3	4.5×10^{-8}
			7	4.4×10^{-8}
		12	3	3.3×10^{-8}
			7	3.2×10^{-8}
	64	6	3	3.5×10^{-8}
			7	3.4×10^{-8}
		12	3	2.3×10^{-5}
			7	2.8×10^{-7}
	128	6	3	1.3×10^{-5}
			7	1.4×10^{-6}
		12	3	1.0×10^{-5}
			7	3.1×10^{-5}

$\beta = 2.5 \times 10^{-8}$ was selected as a well-balanced trade-off between reconstruction accuracy and regularization strength.

B.2 Performance prediction network

The number of hidden layers, the number of hidden nodes, and the activation function were selected as the hyperparam-

eters. Tables 6 and 7 summarize the hyperparameter tuning results for two-parameter and multi-parameter cases, respectively. The best model, which achieved the lowest NLL, is highlighted in bold.

Table 6 Hyperparameter tuning results of PPN; two-parameter case with 200 LHS samples

Activation	Number of hidden layer	Number of hidden node	NLL _{DE}
Tanh	2	16	−4.26
		32	−4.20
		64	−2.71
	4	16	−4.30
		32	−3.95
		64	−2.48
	6	16	−4.26
		32	−4.11
		64	−2.41
LeakyReLU	2	16	−3.54
		32	−3.89
		64	−3.90
	4	16	−3.80
		32	−4.02
		64	−0.86
	6	16	−3.62
		32	−3.62
		64	−1.35

Table 7 Hyperparameter tuning results of PPN; multi-parameter case

Activation	Number of hidden layer	Number of hidden node	NLL _{DE}
Tanh	4	16	−2.76
		32	−3.15
		64	−3.48
	6	16	−2.89
		32	−3.30
		64	−3.60
	8	16	−2.95
		32	−3.37
		64	−3.54
ReLU	4	16	−2.72
		32	−3.14
		64	−3.35
	6	16	−2.82
		32	−3.19
		64	−3.36
	8	16	−2.82
		32	−3.18
		64	−3.24

Appendix C Supplementary validation

C.1 Airfoil generation network

To access the generalization capability of trained AGN, additional validation was conducted on both the reconstruction accuracy and the statistical consistency of generated airfoils.

Figure 25 compares the original and reconstructed airfoils for representative cases. The black solid line represents the original airfoil, the red dashed line denotes the reconstructed airfoil that best fits the original, and the blue dotted line indicates the reconstruction error, defined as the difference in the y coordinates. For clearer visualization, the error is magnified ten times along the y-axis.

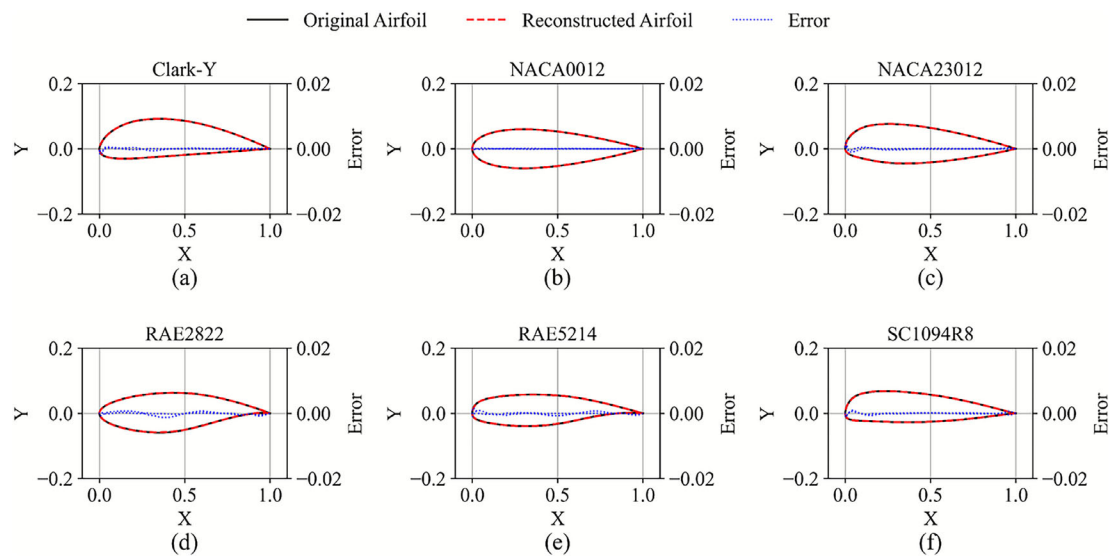


Fig. 25 Reconstruction error of AGN; **a** Clark Y, **b** NACA0012, **c** NACA23012, **d** RAE2822, **e** RAE5214, **f** SC1094R8

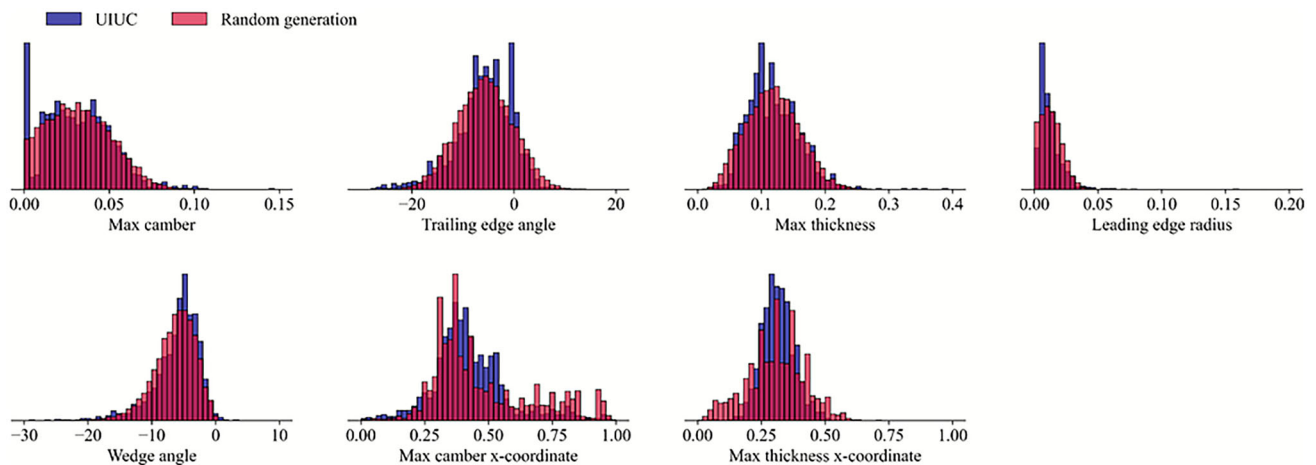


Fig. 26 Histograms of geometric features; UIUC airfoil database (blue) and randomly generated airfoil by AGN (red)

The results show that the AGN can accurately reconstruct the geometric features of the airfoils. While increasing the number of variables could further improve reconstruction quality, it would inevitably degrade optimization efficiency due to the curse of dimensionality problem. Hence, the selected configuration provides a practical balance between accuracy and efficiency.

To further validate the generalization capability of the AGN, the geometric features of the airfoils were computed for both the UIUC airfoil database and the airfoils randomly generated by the AGN. Figure 26 presents the histograms of these features. As shown, the overall distributions of the airfoil features are highly similar. Notably, even though parameters such as the trailing edge wedge angle and the x-coordinates of the maximum thickness and camber were not explicitly informed during the training of the AGN,

their resulting distributions closely match those of the UIUC database. This result demonstrates that the AGN successfully learned the geometric representations of the UIUC airfoil database.

5.1 C.2 Performance prediction network

Figure 27 presents the validation results using a cross-validation plot (left column) and a cumulative distribution of NMAE (right column). The cross-validation plots show a comparison between the aerodynamic coefficients predicted by CFD (x-axis) and those estimated by the PPN (y-axis). In an ideal scenario, all data points would lie on the $y = x$ line, indicating perfect agreement. Deviations from this line reflect prediction errors, with larger errors corresponding to greater distances from the ideal line. To facilitate visual assessment

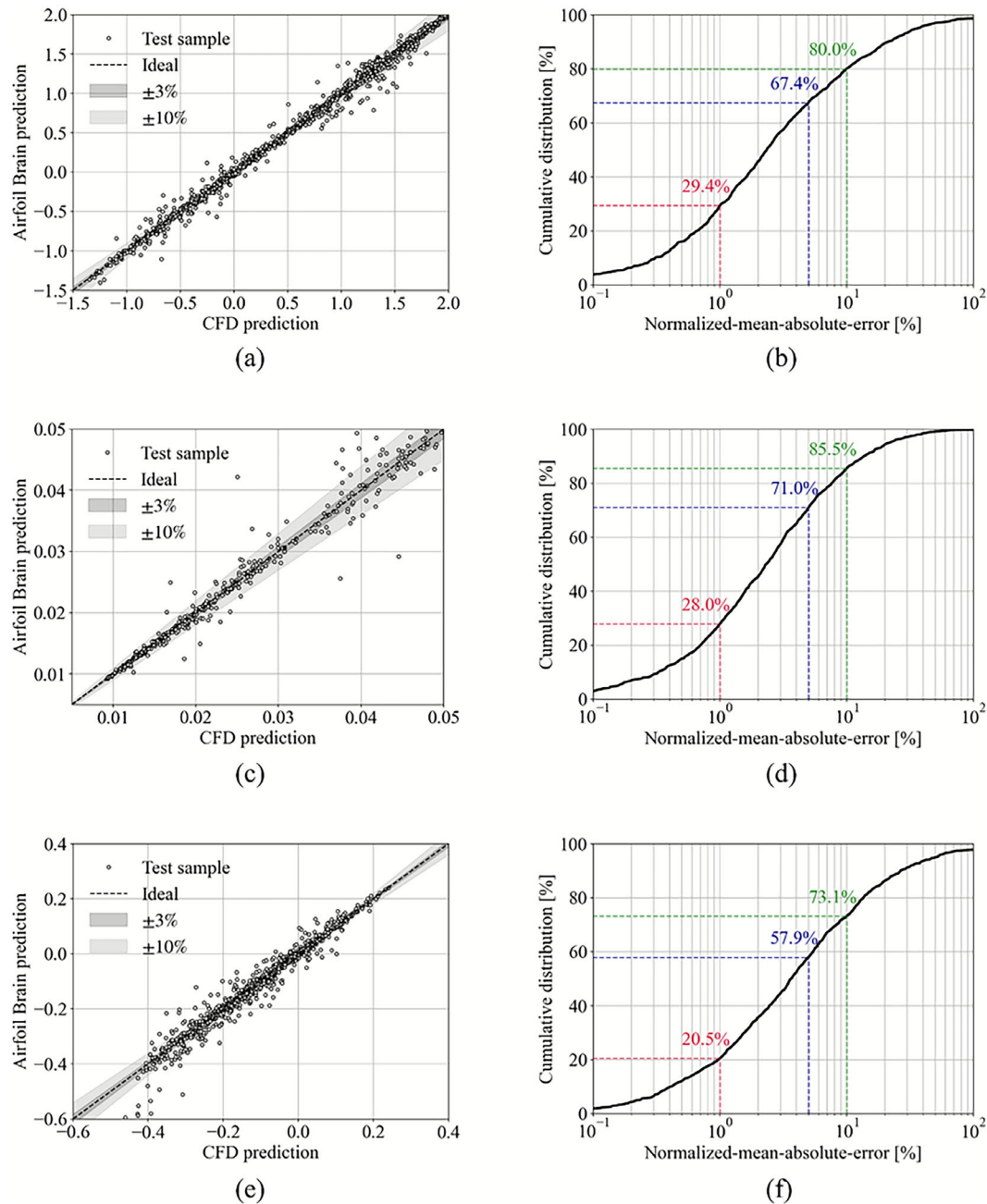


Fig. 27 Validation results of the performance prediction network. Cross-validation plots (left column) and cumulative normalized mean absolute error (NMAE) distributions (right column) are shown for each

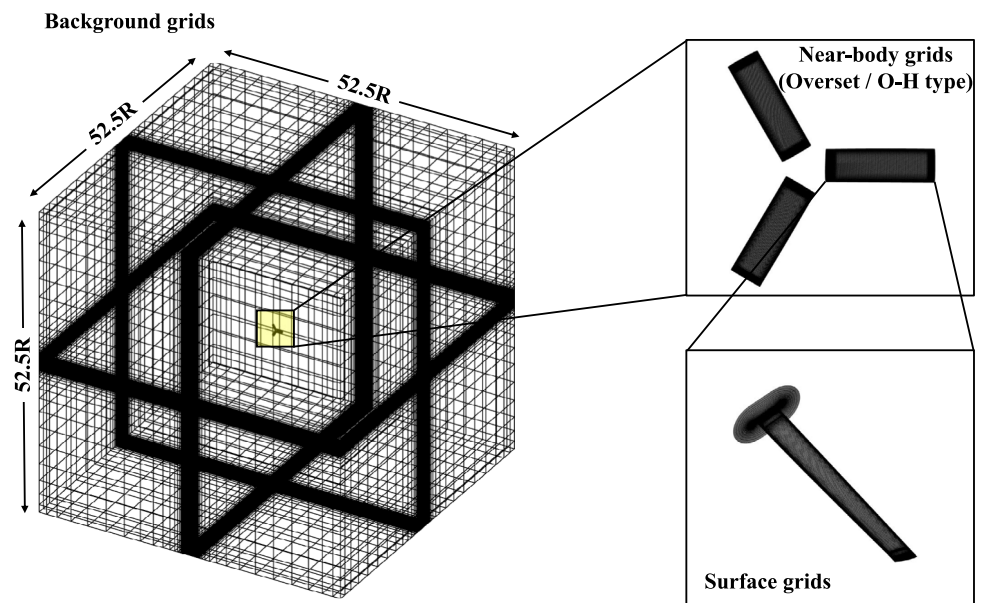
quantity of interest; **a** C_l cross-validation, **b** cumulative NMAE of C_l , **c** C_d cross-validation, **d** cumulative NMAE of C_d , **e** C_m cross-validation, **f** cumulative NMAE of C_m

of prediction accuracy, shaded regions that represent $\pm 3\%$ and $\pm 10\%$ deviations are also plotted. The cumulative distribution plots are presented to statistically assess the overall prediction error of the model. The x-axis represents NMAE on a logarithmic scale and the y-axis represents its cumulative distribution. Vertical dashed lines are drawn at representative NMAE thresholds (e.g., 1%, 5%, and 10%, represented as

red, blue, and green, respectively) to illustrate the proportion of samples falling within each error bound.

Figure 27 shows that the prediction of aerodynamic coefficients has good agreement with that of CFD, with most of the samples distributed near the ideal $y = x$ line. This trend is further supported by the cumulative error plots in Fig. 27b, d, and f. Over 50% of the test samples have an NMAE below

Fig. 28 Computational grid for RANS simulation of the rotor blade: 10.9M cells for the background grid and 3.2M cells for the near-body grid of each blade



5%, and around 70% fall below 10%, consistently across C_l , C_d , and C_m .

Appendix D computational grid systems for three-dimensional RANS simulation of propotor

The computational grids used for conducting RANS simulations on the optimized propotor are presented in Fig. 28. A total of 20.5 million cells is used, consisting of 10.9 million points for the background grids – with a spacing of 0.15 times the mean chord length for the wake region – and 3.2 million points for each blade in the near-body grid. The blade grids adopt an O–H topology with y^+ maintained below 1, and consist of 51, 201, and 261 points in the normal, chordwise, and spanwise directions, respectively.

Acknowledgements We would like to thank Prof. Soo Hyung Park of Konkuk University for providing the KFLOW software, which has greatly contributed to the quality of this work.

Author Contributions Yu-Eop Kang: conceptualization, formal analysis, investigation, methodology, software, validation, visualization, writing – original draft, and writing – review and editing. Dawoon Lee: conceptualization, methodology, writing – review. Yoonpyo Hong: methodology, software, validation, writing – review. Sunwoong Yang: formal analysis, methodology, visualization, writing – review. Kwanjung Yee: funding acquisition, project administration, resources, supervision, writing – review.

Funding This work was supported by the Ministry of Science and ICT of Korea (No. RS-2024-00355857).

Data Availability The source code and datasets supporting the findings of this study are publicly available at: <https://github.com/yueop-kang/airfoil-brain>.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no Conflict of interest.

Replication of results All model settings and computational procedures are described in the manuscript to ensure reproducibility. The dataset and source code are not publicly available due to institutional restrictions.

Ethics approval and consent to participate This study does not involve human participants or animals.

References

- Achour G, Sung WJ, Pinon-Fischer OJ, Mavris DN (2020) Development of a conditional generative adversarial network for airfoil shape optimization. In: AIAA SciTech 2020 Forum, Orlando, Florida
- Anand A, Marepally K, Muneeb Safdar M, Baeder JD (2024) A novel approach to inverse design of wind turbine airfoils using tandem neural networks. *Wind Energy* 27(9):900–921
- Arjovsky M, Chintala S, Bottou L (2017) Wasserstein generative adversarial networks. In: International Conference on Machine Learning, Sydney, Australia, pp. 214–223. PMLR
- Aupoix B, Spalart P (2003) Extensions of the spalart-allmaras turbulence model to account for wall roughness. *Int J Heat Fluid Flow* 24(4):454–462
- Bauchau OA, Bottasso CL, Nikishkov YG (2001) Modeling rotorcraft dynamics with finite element multibody procedures. *Math Comput Model* 33(10–11):1113–1137
- Bishop CM, Nasrabadi NM (2006) Pattern Recognition and Machine Learning, vol 4. Springer, New York

- Blundell C, Cornebise J, Kavukcuoglu K, Wierstra D (2015) Weight uncertainty in neural network. In: International Conference on Machine Learning, pp. 1613–1622. PMLR
- Bouhlef MA, He S, Martins JR (2020) Scalable gradient-enhanced artificial neural networks for airfoil shape design in the subsonic and transonic regimes. *Struct Multidiscip Optim* 61(4):1363–1376
- Chen W, Chiu K, Fuge MD (2020) Airfoil design parameterization and optimization using bézier generative adversarial networks. *AIAA J* 58(11):4723–4735
- Deb K, Agrawal S, Pratap A, Meyarivan T (2000) A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii. In: Parallel Problem Solving from Nature PPSN VI: 6th International Conference Paris, France, September 18–20, 2000 Proceedings 6, pp. 849–858. Springer
- Derksen R, Rogalsky T (2010) Bezier-parsec: an optimized aerofoil parameterization for design. *Adv Eng Softw* 41(7–8):923–930
- Droandi G, Gibertini G (2015) Aerodynamic blade design with multi-objective optimization for a tiltrotor aircraft. *Aircraft Eng Aerospace Technol Int J* 87(1):19–29
- Du X, He P, Martins JR (2021) Rapid airfoil design optimization via neural networks-based parameterization and surrogate modeling. *Aerospace Sci Technol* 113:106701
- Du X, He P, Martins JR (2020) A b-spline-based generative adversarial network model for fast interactive airfoil aerodynamic optimization. In: AIAA SciTech 2020 Forum, Orlando, Florida
- Fuhg JN, Fau A, Nackenhorst U (2021) State-of-the-art and comparative review of adaptive sampling methods for kriging. *Archiv Comput Methods Eng* 28:2689–2747
- Gal Y, Ghahramani Z (2016) Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In: International Conference on Machine Learning, pp. 1050–1059. PMLR
- Gilkeson C, Toropov V, Thompson H, Wilson M, Foxley N, Gaskell P (2014) Dealing with numerical noise in cfd-based design optimization. *Comput Fluids* 94:84–97
- Goldstein S (1929) On the vortex theory of screw propellers. *Proc Royal Soc London Ser A Cont Papers Math Phys Character* 123(792):440–465
- Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2020) Generative adversarial networks. *Commun ACM* 63(11):139–144
- Guo C, Pleiss G, Sun Y, Weinberger KQ (2017) On calibration of modern neural networks. In: International Conference on Machine Learning, pp. 1321–1330. PMLR
- Han X, Sagaut P, Lucor D (2012) On sensitivity of rans simulations to uncertain turbulent inflow conditions. *Comput Fluids* 61:2–5
- Healy RC, Beals N, Anusonti-Inthra P (2024) Multi-fidelity training data generation for the development of deepairfoil, an airfoil design and analysis tool. In: AIAA Aviation Forum and Ascend 2024
- Higgins I, Matthey L, Pal A, Burgess C, Glorot X, Botvinick M, Mohamed S, Lerchner A (2016) beta-vae: Learning basic visual concepts with a constrained variational framework. In: International Conference on Learning Representations, Toulon, France
- Hodges DH, Saberi H, Ormiston RA (2007) Development of nonlinear beam elements for rotorcraft comprehensive analyses. *J Am Helicopter Soc* 52(1):36–48
- Hong Y, Lee D, Yee K, Park SH (2022) Enhanced high-order scheme for high-resolution rotorcraft flowfield analysis. *AIAA J* 60(1):144–159
- Hong Y, Park SH, Yee K (2023) Comparative assessment of local accuracy of high-order spatial schemes for rotorcraft aeroacoustics. *AIAA J* 61(1):355–377
- Hong Y, Lee D, Kang Y-E, Yee K (2024) Rotor blade design optimization with airfoil consideration using advanced reduced order models. In: Vertical Flight Society Forum
- Jain RK, Yeo H, Ho JC, Bhagwat M (2016) An assessment of rcas performance prediction for conventional and advanced rotor configurations. *J Am Helicopter Soc* 61(4):1–12
- Johnson, W (1998) Rotorcraft aeromechanics applications of a comprehensive analysis. In: Heli Japan, 98:21–23
- Kallstrom K, Shirazi D (2024) Airfoil table generation and validation for the vr-12 and ssc-a09 airfoils and quadrotor performance prediction. In: 2024 Transformative Vertical Flight Meeting. NASA Ames Research Center, Santa Clara, CA, USA. The Vertical Flight Society (VFS)
- Kang Y-E, Lee D, Yee K (2023) Compact and intuitive airfoil parameterization method through physics-aware variational autoencoder. arXiv preprint [arXiv:2311.10921](https://arxiv.org/abs/2311.10921)
- Kang Y-E, Lee D, Yee K (2024) Physically interpretable airfoil parameterization using variational autoencoder-based generative modeling. In: AIAA SciTech 2024 Forum
- Kendall A, Gal Y (2017) What uncertainties do we need in bayesian deep learning for computer vision?. *Adv Neural Inf Proc Syst*. 30
- Kim KH, Kim C, Rho O-H (2001) Methods for the accurate computations of hypersonic flows: I. ausmpw+ scheme. *J Comput Phys* 174(1):38–80
- Kingma DP (2014) Adam: A method for stochastic optimization. arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980)
- Kingma DP, Welling M (2013) Auto-encoding variational bayes. arXiv preprint [arXiv:1312.6114](https://arxiv.org/abs/1312.6114)
- Koren B (1993) A robust upwind discretization method for advection, diffusion and source terms. Technical report, Centrum Wiskunde & Informatica (CWI)
- Kulfan BM (2008) Universal parametric geometry representation method. *J Aircr* 45(1):142–158
- Kyprioti AP, Zhang J, Taflanidis AA (2020) Adaptive design of experiments for global kriging metamodeling through cross-validation information. *Struct Multidiscip Optim* 62(3):1135–1157
- Lakshminarayanan B, Pritzel A, Blundell C (2017) Simple and scalable predictive uncertainty estimation using deep ensembles. *Adv Neural Inf Proc Syst*. 30
- Leishman JG, Rosen KM (2011) Challenges in the aerodynamic optimization of high-efficiency propellers. *J Am Helicopter Soc* 56(1):12004
- Li J, Zhang M (2021) Data-based approach for wing shape design optimization. *Aerospace Sci Technol* 112:106639
- Li J, Bouhlef MA, Martins JR (2019) Data-based approach for fast airfoil analysis and optimization. *AIAA J* 57(2):581–596
- Liu H, Xu S, Ma Y, Chen X, Wang X (2016) An adaptive Bayesian sequential sampling approach for global metamodeling. *J Mech Des* 138(1):011404
- Maisel MD (2000) The History of the XV-15 Tilt Rotor Research Aircraft: from Concept to Flight, vol 17. National Aeronautics and Space Administration, Washington, DC
- Masters DA, Taylor NJ, Rendall T, Allen CB, Poole DJ (2017) Geometric comparison of aerofoil shape parameterization methods. *AIAA J* 55(5):1575–1589
- Ning SA (2014) A simple solution method for the blade element momentum equations with guaranteed convergence. *Wind Energy* 17(9):1327–1345
- Park SH, Kwon JH (2004) Implementation of kw turbulence models in an implicit multigrid method. *AIAA J* 42(7):1348–1357
- Piegl L, Tiller W (1997) The NURBS Book, 2nd edn. Monographs in Visual Communication. Springer, Berlin, Heidelberg
- Rasmussen CE (2003) Gaussian processes in machine learning. Springer, Berlin, Heidelberg
- Saberi H, Khoshlahjeh M, Ormiston RA, Rutkowski MJ (2004) Overview of rcas and application to advanced rotorcraft problems. In: American Helicopter Society 4th Decennial Specialists' Conference on Aeromechanics, San Francisco, CA

- Safdar MM, Anand A, Marepally K, Lee B, Baeder JD (2024) A surrogate modeling framework for airfoil design and optimization. In: AIAA SciTech 2024 Forum
- Savitzky A, Golay MJ (1964) Smoothing and differentiation of data by simplified least squares procedures. *Anal Chem* 36(8):1627–1639
- Selig M (1996) UIUC Airfoil Data Site. University of Illinois Urbana-Champaign, United States. <https://m-selig.ae.illinois.edu/ads.html>
- Shalu H, Govindarajan B, Sridharan A, Singh R (2023) Blade shape optimization of rotors using neural networks. In: Vertical Flight Society Forum 79, vol. 79
- Sobieczky H (1999). In: Fujii K, Dulikravich GS (eds) *Parametric Airfoils and Wings*. Vieweg+Teubner Verlag, Wiesbaden
- Spalart P, Allmaras S (1992) A one-equation turbulence model for aerodynamic flows. In: 30th Aerospace Sciences Meeting and Exhibit, p. 439
- Toal DJ, Bressloff NW, Keane AJ, Holden CM (2010) Geometric filtration using proper orthogonal decomposition for aerodynamic design optimization. *AIAA J* 48(5):916–928
- Xie H, Wang J, Zhang M (2024) Parametric generative schemes with geometric constraints for encoding and synthesizing airfoils. *Eng Appl Artif Intel* 128:107505
- Yang S, Yee K (2024) Towards reliable uncertainty quantification via deep ensemble in multi-output regression task. *Eng Appl Artif Intel* 132:7871
- Yang S, Lee S, Yee K (2023) Inverse design optimization framework via a two-step deep learning approach: application to a wind turbine airfoil. *Eng Computers* 39(3):2239–2255
- Yilmaz E, German B (2020) Conditional generative adversarial network framework for airfoil inverse design. In: AIAA Aviation 2020 Forum, Online

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.